

Week 3: Language Modelling, GPT, Fine-tuning & Decoding Strategies

Comprehensive Study Notes with Derivations, Diagrams & Practice

Compiled by Deon Dmello
IITM BS in Programming & Data Science

September 22, 2026

Source: Based on Lecture 2 – *Language Modelling, GPT, Decoding Strategies* (Mitesh M. Khapra, AI4Bharat, Dept. of CSE, IIT Madras). These notes cover Modules 2.1–2.4 of the deck (Language Modelling → GPT architecture → Pre-training and Fine-tuning → Decoding Strategies), with every formula re-derived with explicit tensor-dimension bookkeeping, intuition boxes, and reconstructed TikZ diagrams for every architecture figure in the slides – plus extra intuition/notes beyond the slides where they help build a complete picture (clearly marked).

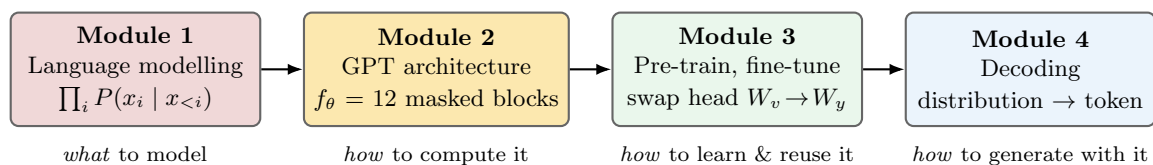
Contents

Roadmap and Notation	3
1 Language Modelling	4
1.1 Recap: The Transformer as a Black Box	4
1.2 The Pre-training → Fine-tuning Idea	4
1.3 Formalizing a Language Model	5
1.3.1 The Chain Rule Decomposition	5
1.3.2 Autoregressive Models and Causal Language Modelling (CLM)	6
1.4 Three Ways to Realize f_θ with a Transformer	6
2 Generative Pre-trained Transformer (GPT)	8
2.1 Stacking Transformer Blocks	8
2.2 Inside a Transformer (GPT) Block	8
2.2.1 Masked Multi-Head Self-Attention – Full Derivation	10
2.2.2 Feed-Forward Network – Full Derivation	11
2.3 Pre-training Corpus: BookCorpus	13
2.4 GPT-1 Architecture Hyperparameters	13
2.5 Counting GPT-1's Parameters	13
2.6 Following One Batch Through GPT-1: Every Tensor Shape	15
3 Pre-training and Fine-tuning	16
3.1 Pre-training Objective and Procedure	16
3.2 Fine-tuning: Adapting GPT to Downstream Tasks	17
3.3 Case Studies: Formatting Inputs for Different Tasks	18
3.3.1 (i) Sentiment Analysis – Single-Sequence Classification	18
3.3.2 (ii) Textual Entailment / Contradiction – Sentence-Pair Classification	18
3.3.3 (iii) Multiple Choice – Shared Encoder, Per-choice Heads	19
3.3.4 (iv) Text Generation – Autoregressive Decoding	19

3.4	Wishlist for Text Generation	20
4	Decoding Strategies	22
4.1	Exhaustive Search	22
4.2	Greedy Search	23
4.3	Beam Search	25
4.4	Sampling Strategies	27
4.4.1	Top- K Sampling	27
4.4.2	Choosing K is Hard: the Flat vs. Peaked Problem	27
4.4.3	Solution 1: Temperature Sampling	28
4.4.4	Solution 2: Top- p (Nucleus) Sampling	29
5	One-Page Revision Sheet	31
6	Practice: Test Your Knowledge	32
6.1	Numerical Problems (with Full Worked Solutions)	32
6.2	Subjective Questions (with Full Answers)	36
6.3	Multiple Choice Questions	40
6.4	MCQ Answer Key with Explanations	45

Roadmap and Notation

The one-sentence story of this lecture: write the probability of text as a product of next-token probabilities (Module 1), compute every one of those probabilities with a stack of causally-masked transformer blocks (Module 2), learn the weights once on raw books and then re-use them for labelled tasks by changing only the input format and the output head (Module 3), and finally turn the model's per-step distributions back into actual text with a decoding strategy (Module 4).



Symbol	Meaning	GPT-1 value
$\mathcal{V}, \mathcal{V} $	vocabulary and its size	40,478 (BPE)
T	sequence length / context size	512
B	batch size	64
d_{model} (slides: C , embedding-dim)	width of every hidden vector	768
H	number of attention heads	12
$d_k = d_{\text{model}}/H$	per-head query/key/value width	64
d_{ff}	FFN hidden width	$4 \times 768 = 3072$
n	number of transformer blocks	12
$h_l \in \mathbb{R}^{T \times d_{\text{model}}}$	output of block l (h_0 = embeddings)	–
$h_l[i], h_l^i$	row i of h_l : the vector at position i	$\in \mathbb{R}^{768}$
$W_v \in \mathbb{R}^{d_{\text{model}} \times \mathcal{V} }$	LM head (hidden $\rightarrow$ vocabulary logits)	$768 \times 40,478$
W_y	task head added during fine-tuning	random init
M	causal mask (0 on/below diagonal, $-\infty$ above)	$T \times T$
k	beam width (Module 4)	–
K, p	top- K size, top- p threshold (Module 4)	–
$\mathcal{T}$	softmax temperature (slides write T)	–

Note / Gotcha

Two notational clashes in the slides, resolved here. (1) The slides use T both for sequence length and for softmax temperature; these notes write temperature as $\mathcal{T}$. (2) The slides use W_v for the *vocabulary* projection (LM head) and also write W_V for the attention *value* projection; these notes use W_v only for the LM head and $W_V^{(j)}$ (capital, with a head index) for values.

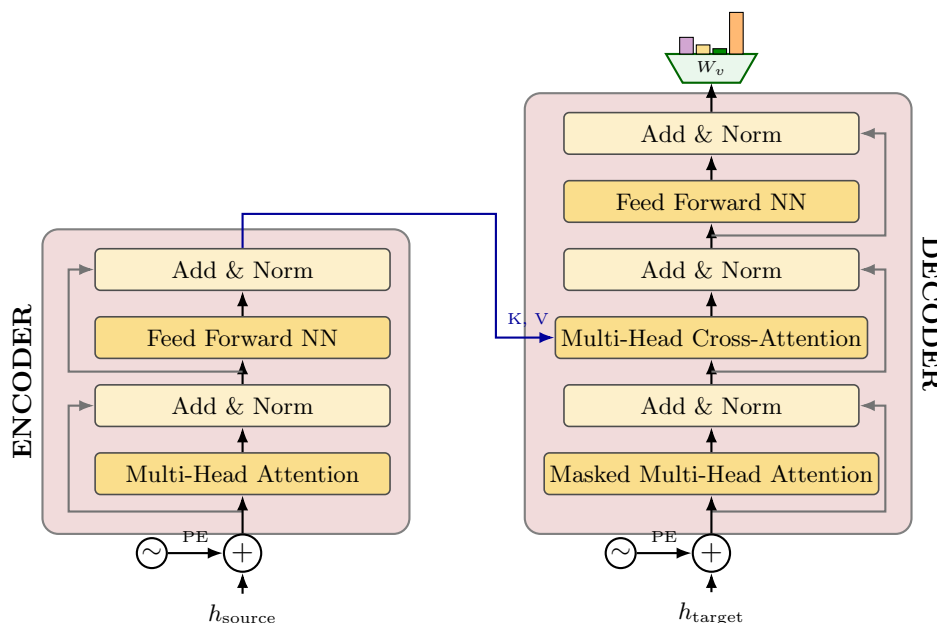
Box legend

- Intuition
- Key Idea
- Note / Gotcha
- Beyond the Slides (not examinable from the deck)
- Derivation (every step, with shapes)
- Worked Example (numbers from the slides)
- Exam Trap (common wrong answers)
- Module Summary

Module 1 Language Modelling

1.1 Recap: The Transformer as a Black Box

In the previous lecture we studied the transformer in the context of machine translation: an **encoder** that reads the source sentence and a **decoder** that autoregressively produces the target sentence, connected by cross-attention.



At an even higher level of abstraction, the whole transformer is simply a black box f_θ : *text in source language* $\rightarrow$ *text in target language*. This works beautifully for translation, but raises a question: **what if we want to use the same architecture for other NLP tasks** – sentiment classification, summarization, question answering, ...?

Intuition

The naive approach is to train a *separate* transformer from scratch for every task, using a dataset labelled specifically for that task. This has two serious problems:

1. Training from random initialization for every task is slow and data-hungry – convergence takes a long time.
2. Large labelled datasets are expensive and laborious to construct, and for many tasks we simply do not have enough labelled examples.

On the other hand, **unlabelled** text is available in near-unlimited quantity on the internet (Wikipedia, the open web, digitized books, blogs, ...). This motivates the central question of this module: *can we exploit this unlabelled text to learn a good starting point for a model, so that it needs only minimal supervision (zero- or few-shot) to adapt to a downstream task?*

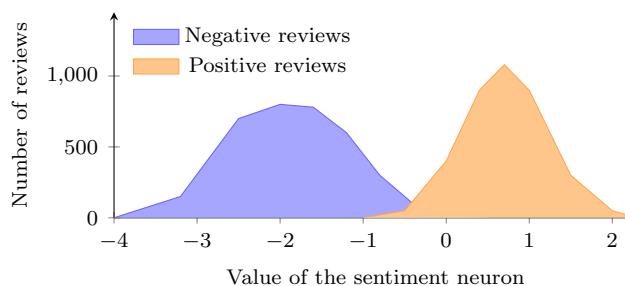
1.2 The Pre-training $\rightarrow$ Fine-tuning Idea

Key Idea

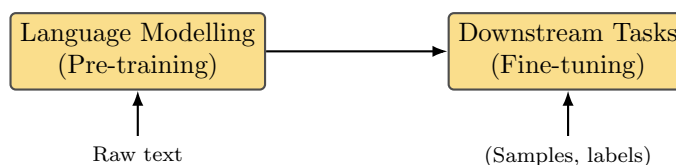
Two-stage recipe.

1. **Pre-training**: expose the model to a huge amount of raw (unlabelled) text and have it learn a generic, task-agnostic “understanding” of language.
2. **Supervised fine-tuning**: starting from the pre-trained parameters, adapt the model to a specific downstream task using a comparatively small labelled dataset.

This idea is not just plausible – it is empirically powerful. Radford et al. (2017) trained a byte-level LSTM language model on Amazon review text and discovered, after training, a *single neuron* in the hidden state whose value alone separated positive from negative reviews:



With this single-neuron representation, a *linear* classifier reached **91.8%** accuracy on sentiment classification, beating the previous state of the art, and matched the performance of fully supervised systems while using **30–100× fewer labelled examples**. This is direct evidence that language modelling on raw text forces a network to build representations that are useful far beyond next-word prediction.



1.3 Formalizing a Language Model

Definition

Let $\mathcal{V}$ be the **vocabulary** of a language – the set of all unique tokens (words/subwords) in that language. A sentence is a sequence $X_1, X_2, \dots, X_n$ with each $X_i \in \mathcal{V}$.

Worked Example

Let $\mathcal{V} = \{\text{an, apple, ate, I}\}$. Some possible sequences (not necessarily grammatical) are:

- (a) An apple ate I (d) an apple
- (b) I ate an apple (e) ...
- (c) I ate apple

Intuitively, sentence (b) is far more *probable* than sentence (a) in natural English – i.e., if we scanned a huge corpus, we would see (b)-like patterns vastly more often than (a)-like ones.

This intuition – “some sequences are more probable than others” – is formalized as follows.

Definition

A **language model** is a function that maps a sequence of tokens to a probability:

$$f : (X_1, X_2, \dots, X_n) \longrightarrow [0, 1].$$

1.3.1 The Chain Rule Decomposition

Directly modelling $P(x_1, \dots, x_T)$ over an exponentially large space of sequences is intractable. The chain rule of probability lets us factor the joint distribution into a product of *conditional* distributions:

Derivation

By repeated application of $P(A, B) = P(A)P(B | A)$:

$$\begin{aligned} P(x_1, x_2, \dots, x_T) &= P(x_1) P(x_2 | x_1) P(x_3 | x_2, x_1) \cdots P(x_T | x_{T-1}, \dots, x_1) \\ &= \prod_{i=1}^T P(x_i | x_1, \dots, x_{i-1}). \end{aligned} \tag{1}$$

This is an **exact** identity – no assumption has been made yet. It reduces the problem of modelling one joint distribution over T tokens to modelling T conditional (“next-token”) distributions.

Note / Gotcha

A tempting but wrong shortcut. If we *naively* assumed the tokens in a sequence were mutually independent, the joint would collapse to a product of unigram (marginal) probabilities:

$$P(x_1, x_2, \dots, x_T) = \prod_{i=1}^T P(x_i) \quad (\text{unigram / bag-of-words model}).$$

This is a much weaker and *incorrect* model of language: consider

(a) “I enjoyed reading a **book**” vs. (b) “I enjoyed reading a **thermometer**”.

The presence of the word “enjoyed” makes “book” far more likely to follow than “thermometer” – but a unigram model, having thrown away all ordering/context information, would assign these two continuations relative probabilities based purely on how often “book” vs. “thermometer” occur *anywhere* in the corpus, ignoring context entirely. Hence the independence assumption does not make sense; we are forced to keep the full conditional form of Eq. (1), where the current word x_i depends on *all* previous words $x_1, \dots, x_{i-1}$.

1.3.2 Autoregressive Models and Causal Language Modelling (CLM)

The next question is: *how do we actually estimate/compute the conditional probabilities* $P(x_i | x_1, \dots, x_{i-1})$? One elegant solution is to use an **autoregressive model**: a single parameterized function f_θ (with a *fixed* number of parameters, regardless of T) that outputs all these conditionals.

Definition

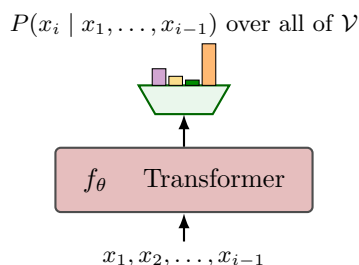
Causal Language Modelling (CLM). We look for parameters θ of a function f_θ such that

$$P(x_i | x_1, \dots, x_{i-1}) = f_\theta(x_i | x_1, \dots, x_{i-1}), \quad \forall i \in [1, T],$$

so that

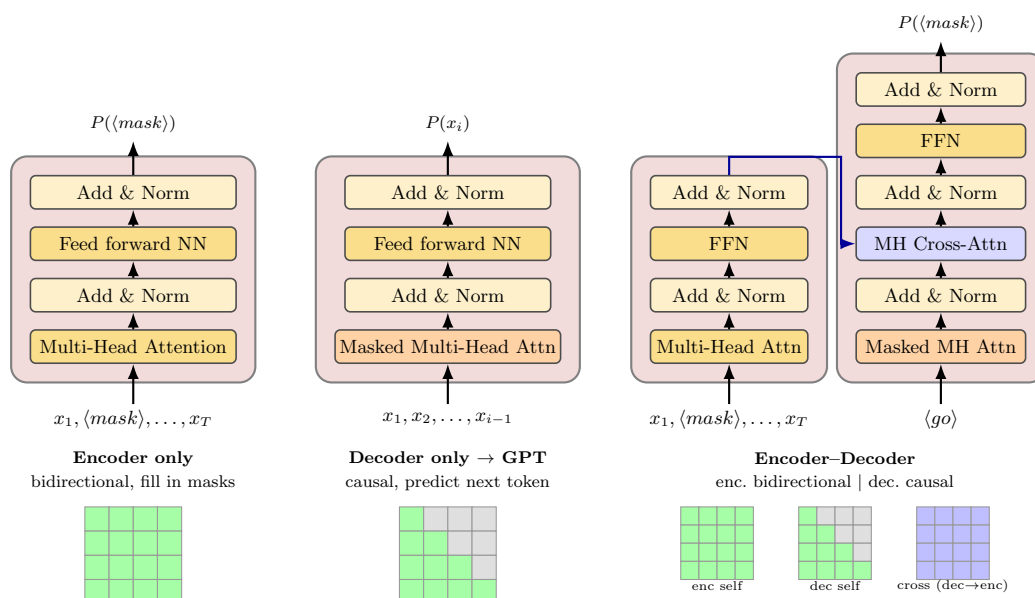
$$P(x_1, \dots, x_T) = \prod_{i=1}^T f_\theta(x_i | x_1, \dots, x_{i-1}).$$

The task of predicting the next token given all previous tokens is called **language modelling**, and can f_θ be a transformer?



1.4 Three Ways to Realize f_θ with a Transformer

The transformer offers three natural architectural choices, depending on which piece(s) of the original encoder–decoder we keep. The row of small grids underneath each architecture shows **which positions may attend to which** (row = query position, column = key position, green = allowed) – this attention pattern is the real difference between the three families.



Intuition

- **Encoder only:** bidirectional self-attention (every token sees every other token). Trained by hiding tokens ($\langle \text{mask} \rangle$) and predicting them from both left and right context – this is a *masked* language model (the BERT family).
- **Decoder only:** causal (masked) self-attention only – token i can see tokens $1, \dots, i$ but not the future. Trained with the CLM objective of Eq. (1) – this is exactly the **GPT** family, the subject of Module 2.
- **Encoder-decoder:** encoder reads a (possibly corrupted) input bidirectionally, decoder generates an output autoregressively while cross-attending to the encoder – used for sequence-to-sequence pre-training objectives (e.g. T5-style span corruption).

The rest of these notes focus on the **decoder-only** branch, which is what GPT uses.

Module Summary

- A language model assigns a probability to a sequence of tokens; by the chain rule this factorizes *exactly* into a product of next-token conditionals – no independence assumption is valid.
- CLM parameterizes these conditionals with a single network f_θ applied at every position.
- A decoder-only transformer, with *causal* (masked) self-attention, is a natural and effective choice for f_θ – this is the architecture GPT adopts.

Module 2 Generative Pre-trained Transformer (GPT)

2.1 Stacking Transformer Blocks

GPT builds f_θ by stacking n modified decoder layers, which the original GPT paper calls **transformer blocks**. “Modified” because, unlike a full seq2seq decoder layer, there is *no encoder* to cross-attend to – so each block contains only *masked self-attention* + *feed-forward network*, each wrapped with a residual connection and layer normalization.

Derivation

Let $X \in \mathbb{R}^{T \times d_{\text{model}}}$ be the (embedded) input sequence, T = sequence length, d_{model} = model/embedding dimension.

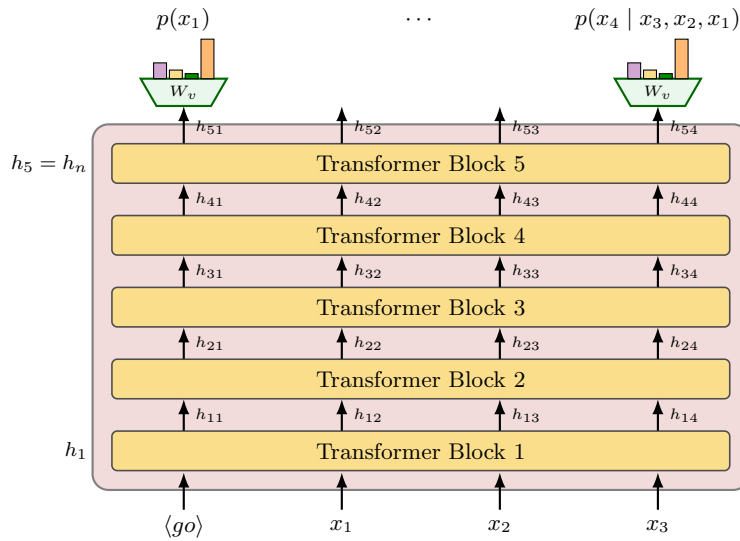
$$h_0 = X \in \mathbb{R}^{T \times d_{\text{model}}}, \quad h_l = \text{transformer_block}(h_{l-1}) \in \mathbb{R}^{T \times d_{\text{model}}}, \quad \forall l \in [1, n].$$

Let $h_n[i] \in \mathbb{R}^{d_{\text{model}}}$ denote the i -th row (output vector at position i) of the final block’s output h_n . The next-token distribution at position i is obtained by projecting into vocabulary space and normalizing:

$$P(x_i) = \text{softmax}(h_n[i] W_v), \quad W_v \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|}, \quad h_n[i] W_v \in \mathbb{R}^{|\mathcal{V}|}.$$

Training maximizes the total log-likelihood (equivalently minimizes cross-entropy) of the corpus:

$$\mathcal{L} = \sum_{i=1}^T \log(P(x_i | x_1, \dots, x_{i-1})).$$

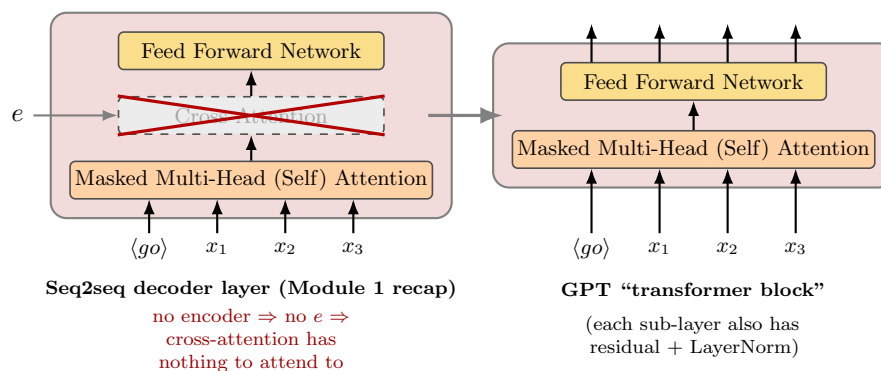


Intuition

Each output column is a next-token distribution conditioned on everything up to (and including) that position: the vector above x_3 produces $P(x_4 | x_3, x_2, x_1)$. This is exactly the chain-rule factorization of Eq. (1), computed for *all* positions in *one parallel forward pass* during training (teacher forcing, Section 3.1), while at inference time it is used one step at a time (Section 3.3.4).

2.2 Inside a Transformer (GPT) Block

The slide starts from the full seq2seq decoder layer (which has three sub-layers) and asks which parts survive when there is no encoder. The masked self-attention is **still required** (to preserve causality); the cross-attention layer is **not needed**, because its keys and values come from the encoder output e , and GPT has no encoder:

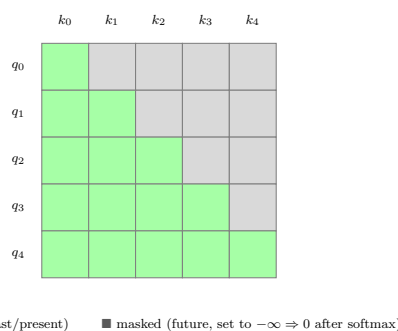


Key Idea

Why causality is essential. We want the model to see only the present and past tokens at every position (never the future – otherwise “predicting” the next word would be trivial cheating by peeking ahead). This is enforced with an additive **mask** M applied to the raw attention scores *before* the softmax. For $T = 4$:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Since $e^{-\infty} = 0$, adding M and then applying softmax *zeroes out* every attention weight pointing to a future position, while leaving present/past positions untouched (adding 0 changes nothing). In practice $-\infty$ is implemented as a very large negative finite number (e.g. -10^9 , or the most negative value the dtype can hold). A causal row is never *fully* masked (the diagonal is always allowed), but once padding masks are combined with it a row can be all $-\infty$, and softmax would then compute $0/0 = \text{NaN}$; a large finite value degrades gracefully instead.



Worked Example

One masked row, by hand. Take query position 2 (of 4). Suppose its scaled scores against keys 1–4 are (2.0, 1.0, 3.0, 0.5). Adding row 2 of M , which is (0, 0, $-\infty$, $-\infty$), gives (2.0, 1.0, $-\infty$, $-\infty$). Softmax:

$$\left(\frac{e^2}{e^2 + e^1}, \frac{e^1}{e^2 + e^1}, 0, 0 \right) = (0.731, 0.269, 0, 0).$$

Note what did *not* happen: position 3 had the highest raw score (3.0), and it still receives exactly zero weight. The mask overrides content; position 2’s output can only mix the value vectors of positions 1 and 2.

Derivation

Why divide by $\sqrt{d_k}$? (the “Scale” box in the slide). Suppose the components of a query q

and a key k are independent with mean 0 and variance 1. Then

$$q \cdot k = \sum_{m=1}^{d_k} q_m k_m, \quad \mathbb{E}[q_m k_m] = 0, \quad \text{Var}(q_m k_m) = \mathbb{E}[q_m^2] \mathbb{E}[k_m^2] = 1,$$

so $\text{Var}(q \cdot k) = d_k$ and its standard deviation is $\sqrt{d_k}$. For GPT-1, $d_k = 64$, so raw scores typically have spread ≈ 8 . Feeding numbers that far apart into softmax pushes it into saturation: one weight ≈ 1 , the rest ≈ 0 , and the gradient through softmax almost vanishes. Dividing by $\sqrt{d_k}$ restores unit variance:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1.$$

2.2.1 Masked Multi-Head Self-Attention – Full Derivation

Derivation

Let the input to the block be $h_{l-1} \in \mathbb{R}^{T \times d_{\text{model}}}$, and suppose we use H attention heads, with per-head dimension $d_k = d_{\text{model}}/H$. For head $j = 1, \dots, H$:

$$Q_j = h_{l-1} W_Q^{(j)}, \quad W_Q^{(j)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad Q_j \in \mathbb{R}^{T \times d_k} \quad (2)$$

$$K_j = h_{l-1} W_K^{(j)}, \quad W_K^{(j)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad K_j \in \mathbb{R}^{T \times d_k} \quad (3)$$

$$V_j = h_{l-1} W_V^{(j)}, \quad W_V^{(j)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad V_j \in \mathbb{R}^{T \times d_k} \quad (4)$$

Raw (masked, scaled) attention scores and weights:

$$S_j = \frac{Q_j K_j^\top}{\sqrt{d_k}} + M \in \mathbb{R}^{T \times T}, \quad A_j = \text{dropout}(\text{softmax}(S_j)) \in \mathbb{R}^{T \times T} \text{ (row-stochastic).}$$

Weighted sum of values, per head:

$$O_j = A_j V_j \in \mathbb{R}^{T \times d_k}.$$

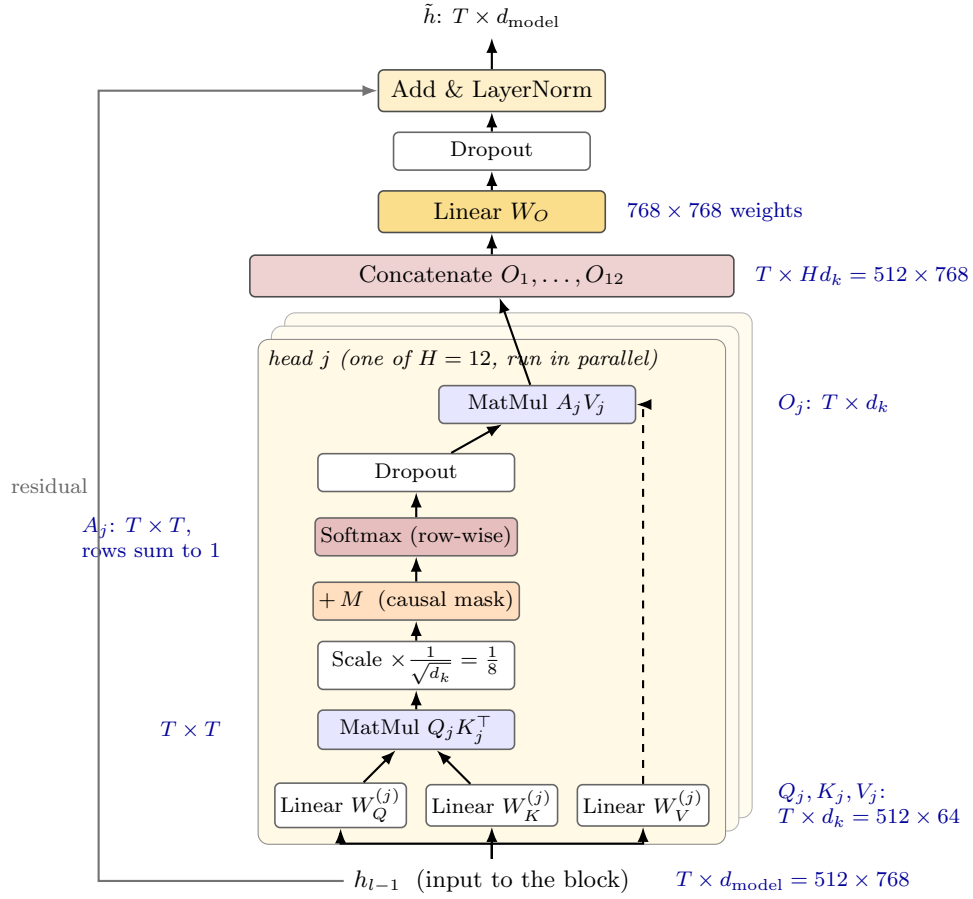
Concatenate all H heads along the feature axis and mix with an output projection:

$$O = \text{Concat}(O_1, \dots, O_H) \in \mathbb{R}^{T \times H d_k} = \mathbb{R}^{T \times d_{\text{model}}},$$

$$O' = \text{dropout}(O W_O) \in \mathbb{R}^{T \times d_{\text{model}}}, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.$$

Residual connection + layer normalization (“Add & Norm”):

$$\tilde{h} = \text{LayerNorm}(h_{l-1} + O') \in \mathbb{R}^{T \times d_{\text{model}}}.$$



Note / Gotcha

The slide draws the first box as “MatMul: $Q^\top K + M$ ” and then “Scale”. Scaling and masking commute here: $(QK^\top + M)/\sqrt{d_k}$ and $QK^\top/\sqrt{d_k} + M$ give the same softmax, because $-\infty/8$ is still $-\infty$ and $0/8$ is still 0. The slide’s $Q^\top K$ is the column-vector convention; with row-vector matrices as used here it is $QK^\top$.

2.2.2 Feed-Forward Network – Full Derivation

Derivation

Each position’s vector $z \in \mathbb{R}^{d_{\text{model}}}$ (a row of $\tilde{h}$) is passed independently (position-wise) through a 2-layer MLP that first *expands* to a wider hidden dimension d_{ff} (typically $d_{\text{ff}} = 4d_{\text{model}}$) and then *projects back*:

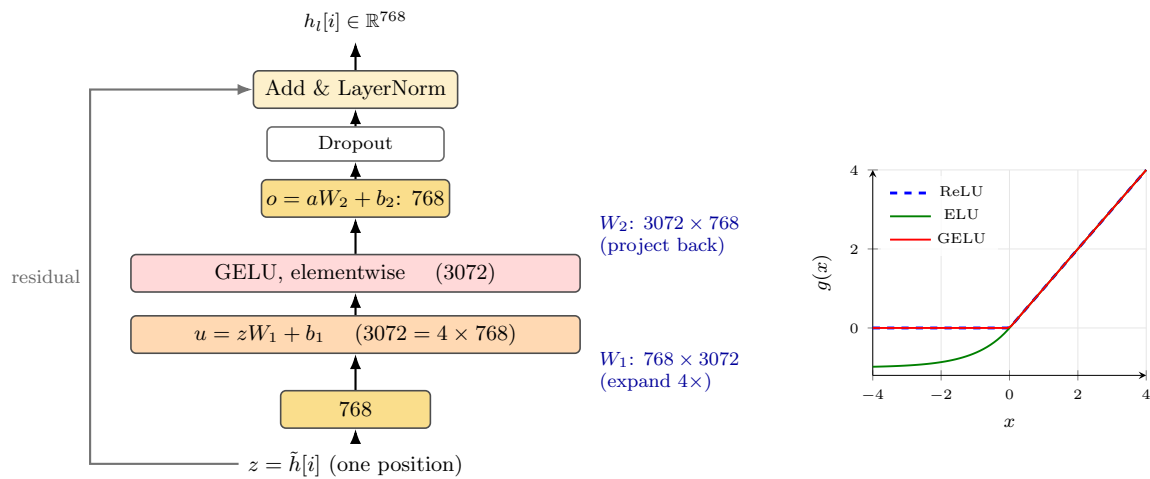
$$u = zW_1 + b_1, \quad W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad b_1 \in \mathbb{R}^{d_{\text{ff}}}, \quad u \in \mathbb{R}^{d_{\text{ff}}} \quad (5)$$

$$a = \text{GELU}(u), \quad a \in \mathbb{R}^{d_{\text{ff}}} \quad (6)$$

$$o = aW_2 + b_2, \quad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad b_2 \in \mathbb{R}^{d_{\text{model}}}, \quad o \in \mathbb{R}^{d_{\text{model}}} \quad (7)$$

followed by dropout and another residual + layer norm:

$$h_l[\text{position}] = \text{LayerNorm}(\tilde{h}[\text{position}] + \text{dropout}(o)) \in \mathbb{R}^{d_{\text{model}}}.$$



Worked Example

GELU vs ReLU at a few points (using $\text{GELU}(x) = x \Phi(x)$):

x	-2	-1	-0.5	1	2
$\Phi(x)$	0.023	0.159	0.309	0.841	0.977
$\text{GELU}(x)$	-0.045	-0.159	-0.154	0.841	1.954
$\text{ReLU}(x)$	0	0	0	1	2

GELU lets small negative inputs through as small negative outputs (minimum ≈ -0.17 near $x \approx -0.75$) and slightly shrinks small positive ones. For large $|x|$ it matches ReLU.

Note / Gotcha

The slide’s FFN figure labels the hidden layer “ $l = 3042$ ”. That is a typo: the hyperparameter slide (and GPT-1 itself) uses $768 \times 4 = \mathbf{3072}$, and every parameter count in Section 2.5 depends on 3072. Also note the FFN is applied *position-wise*: the same W_1, W_2 are applied to every row independently, so it never mixes information across positions. All mixing across positions happens in attention.

Intuition

Why GELU instead of ReLU? $\text{ReLU}(x) = \max(0, x)$ hard-gates: it is either fully “off” (0) or fully “on” (identity), with a non-differentiable kink at 0. $\text{GELU}(x) = x \Phi(x)$, where Φ is the standard normal CDF, can be read as *stochastically gating* x by whether a standard normal draw is $\leq x$, in expectation – it is smooth everywhere, non-monotonic near the origin (it dips slightly negative, like ELU), and this smoothness empirically improves gradient flow and optimization stability in deep transformer stacks. GPT-1 uses GELU; the common closed-form approximation is

$$\text{GELU}(x) \approx 0.5 x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715 x^3) \right] \right).$$

2.3 Pre-training Corpus: BookCorpus

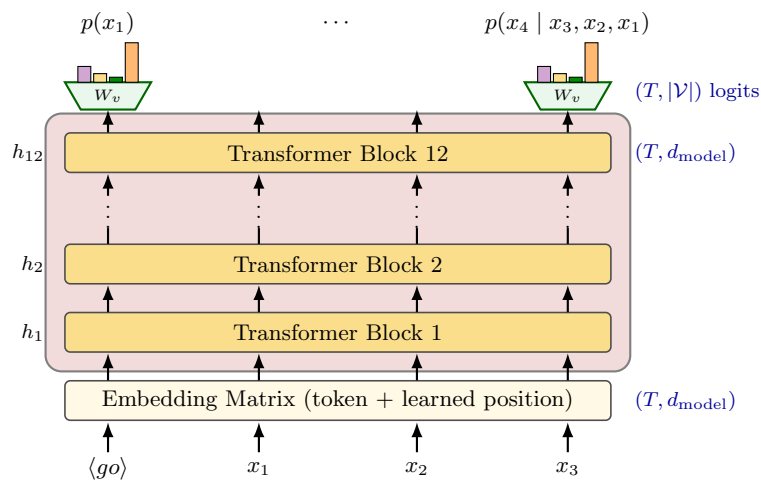
Property	Value
Unique books	7,000
Sentences	74 Million
Words	≈ 1 Billion
Genres	16
Text structure	Long-range <i>contiguous</i> text (no shuffling of sentences/- paragraphs)
Tokenizer	Byte Pair Encoding (BPE)
Vocabulary size $ \mathcal{V} $	40,478
Embedding dimension d_{model}	768

Note / Gotcha

Why contiguous text matters. An alternative benchmark, the *1B Word Benchmark*, is also large, but its sentences are *shuffled* and hence not contiguous (no natural cross-sentence continuation / entailment signal). BookCorpus’s long, unbroken passages let the model learn **long-range dependencies** across sentences and paragraphs – exactly the kind of structure needed for coherent generation and for downstream tasks like entailment (Section 3.3.2).

2.4 GPT-1 Architecture Hyperparameters

Hyperparameter	GPT-1 value
Decoder (transformer) blocks, n	12
Context size, T	512
Attention heads, H	12 ($\Rightarrow d_k = 768/12 = 64$)
Embedding dim, d_{model}	768
FFN hidden size, d_{ff}	$768 \times 4 = 3072$
Activation	GELU
Regularization	Dropout, LayerNorm, residual connections (for faster, more stable convergence)
Positional embeddings	Learned (not fixed sinusoidal, unlike the original Transformer)



2.5 Counting GPT-1's Parameters

We now derive, term by term and with full dimension bookkeeping, the $\approx 117\text{M}$ parameters of GPT-1.

Derivation

(a) **Token embedding matrix.** Shape $|\mathcal{V}| \times d_{\text{model}}$:

$$40,478 \times 768 = 31,087,104 \approx 31 \times 10^6 = 31\text{M (slide rounding)}.$$

(b) **Positional embedding matrix (learned).** Shape (context length) $\times d_{\text{model}}$:

$$512 \times 768 = 393,216 \approx 0.39\text{M (the slide rounds this to 0.3M)}.$$

Embedding total $\approx 31\text{M} + 0.3\text{M} = \mathbf{31.3\text{M}}$ (slide’s rounded figure; the precise sum is 31,480,320 $\approx$ 31.48M).

(c) **Attention parameters, per block.** With $d_k = d_{\text{model}}/H = 768/12 = 64$, each head has three projection matrices $W_Q, W_K, W_V \in \mathbb{R}^{768 \times 64}$:

$$\underbrace{768 \times 64}_{\text{one matrix}} = 49,152 \text{ params,} \quad \underbrace{3 \times 49,152}_{Q,K,V \text{ for one head}} = 147,456 \approx 147 \times 10^3.$$

For all $H = 12$ heads:

$$12 \times 147,456 = 1,769,472 \approx 1.7\text{M}.$$

Plus the output-mixing linear layer $W_O \in \mathbb{R}^{768 \times 768}$:

$$768 \times 768 = 589,824 \approx 0.6\text{M}.$$

Attention params per block $\approx 1.7\text{M} + 0.6\text{M} = 2.3\text{M}$. For all 12 blocks:

$$12 \times 2.3\text{M} = \mathbf{27.6\text{M}}.$$

(d) **Feed-forward parameters, per block.** $W_1 \in \mathbb{R}^{768 \times 3072}$, $b_1 \in \mathbb{R}^{3072}$, $W_2 \in \mathbb{R}^{3072 \times 768}$, $b_2 \in \mathbb{R}^{768}$:

$$2 \times (768 \times 3072) + 3072 + 768 = 4,718,592 + 3,840 = 4,722,432 \approx 4.7\text{M}.$$

For all 12 blocks:

$$12 \times 4.7\text{M} = \mathbf{56.4\text{M}}.$$

Layer	Parameters (Millions)
Embedding layer (token + position)	31.3
Attention layers (all 12 blocks)	27.6
FFN layers (all 12 blocks)	56.4
Total (exact, unrounded)	116.461056

Note / Gotcha

If instead of rounding at each intermediate step we carry *exact* integer counts throughout – token emb. 31,087,104 + pos. emb. 393,216 + attention 28,311,552 ($= 12 \times 2,359,296$) + FFN 56,669,184 ($= 12 \times 4,722,432$) – the grand total is exactly

$$31,087,104 + 393,216 + 28,311,552 + 56,669,184 = 116,461,056 \approx 117 \text{ Million},$$

which is precisely the widely quoted **“GPT-1 has $\approx 117\text{M}$ parameters”**. The slide’s step-by-step rounding ($31.3+27.6+56.4=115.3\text{M}$) is a convenient approximation for mental arithmetic; the exact computation is what actually matches the reported model size.

Embeddings 31.5M (27.0%)	Attention 28.3M (24.3%)	FFN 56.7M (48.7%)
--------------------------	-------------------------	-------------------

Where GPT-1's parameters live (exact counts). Per block, the FFN holds twice as many weights as attention: $8d^2$ vs. $4d^2$.

Exam Trap

Why is there no $768 \times 40,478$ term for the output head W_v ? If W_v were a separate matrix it would add another 31M parameters. GPT-1 *ties* the output projection to the token-embedding matrix: the logits are computed as $h_n[i] W_e^\top$, re-using the same $40,478 \times 768$ matrix W_e that embeds the input tokens. So the LM head costs zero extra parameters, which is why the slide's count stops at embeddings + attention + FFN.

Beyond the Slides

What the slide's count leaves out, and a rule of thumb.

- *Biases in attention:* b_Q, b_K, b_V (3×768) and b_O (768) per block = $3,072 \times 12 = 36,864$.
- *LayerNorm:* each block has 2 LayerNorms, each with a scale γ and shift β of size 768, so $4 \times 768 = 3,072$ per block = 36,864 total.
- Adding both: $116,461,056 + 36,864 + 36,864 = 116,534,784$. Either way the answer rounds to ≈ 117 M.
- *Rule of thumb:* with $d_{\text{ff}} = 4d_{\text{model}}$, each block has $4d^2$ attention weights (Q, K, V, O) and $8d^2$ FFN weights, i.e. $\approx 12d^2$. So the non-embedding parameters are $\approx 12 n d^2 = 12 \times 12 \times 768^2 \approx 84.9$ M, matching $28.3 + 56.7 = 85.0$ M. The same formula estimates much larger models too (e.g. $n = 96$, $d = 12,288$ gives ≈ 174 B, the order of GPT-3's 175B).

2.6 Following One Batch Through GPT-1: Every Tensor Shape

The slides give the input shape $(B, T, C) = (64, 512, 768)$. Here is every intermediate shape for one pre-training step, so the dimension annotations above connect into one pipeline.

#	Step	Operation	Shape (GPT-1)
1	token ids	integers from BPE	$(B, T) = (64, 512)$
2	token embeddings	row lookup in $W_e \in \mathbb{R}^{40478 \times 768}$	$(64, 512, 768)$
3	+ position embeddings	add $W_p[0:T] \in \mathbb{R}^{512 \times 768}$ (broadcast over B)	$(64, 512, 768) = h_0$
4	Q, K, V per block	$h_{l-1} W_Q$, etc., split into heads	$(B, H, T, d_k) = (64, 12, 512, 64)$
5	attention scores	$QK^\top / \sqrt{d_k} + M$	$(B, H, T, T) = (64, 12, 512, 512)$
6	attention weights A	row-wise softmax, dropout	$(64, 12, 512, 512)$
7	per-head output	AV	$(64, 12, 512, 64)$
8	concatenate heads	merge H and d_k axes	$(64, 512, 768)$
9	output proj. + Add&Norm	LayerNorm($h_{l-1} + \text{drop}(OW_O)$)	$(64, 512, 768)$
10	FFN expand	$\text{GELU}(\tilde{h} W_1 + b_1)$	$(64, 512, 3072)$
11	FFN contract + Add&Norm	LayerNorm($\tilde{h} + \text{drop}(a W_2 + b_2)$)	$(64, 512, 768) = h_l$
12	repeat 4–11	for $l = 1, \dots, 12$	$(64, 512, 768) = h_{12}$
13	logits	$h_{12} W_e^\top$ (tied head)	$(B, T, \mathcal{V}) = (64, 512, 40478)$
14	probabilities	softmax over last axis	$(64, 512, 40478)$
15	loss	cross-entropy vs. next tokens, sum/mean	scalar

Beyond the Slides

Two memory facts hiding in this table. (1) Step 5 is the only tensor that grows with T^2 : $64 \times 12 \times 512 \times 512 \approx 201$ M numbers *per block*. Doubling the context quadruples it, which is why long context is expensive for attention. (2) Step 13 is the largest single activation: $64 \times 512 \times 40,478 \approx 1.33$ billion logits, about 5.3 GB in 32-bit floats. The vocabulary size, not the model width, dominates the output layer's memory.

Module Summary

- GPT stacks n *decoder-only* transformer blocks (masked self-attention + FFN, no cross-attention).

- Causal masking via an additive $-\infty$ mask before softmax is what makes the architecture autoregressive.
- GELU (smooth, stochastic-gate interpretation) is used instead of ReLU.
- Positional embeddings are *learned*, unlike the original Transformer’s fixed sinusoidal encoding.
- GPT-1’s $\approx 117\text{M}$ parameters break down as embeddings ($\approx 31\text{M}$) + attention ($\approx 28\text{M}$) + FFN ($\approx 57\text{M}$), with the FFN block being the single largest contributor per layer.

Module 3 Pre-training and Fine-tuning

3.1 Pre-training Objective and Procedure

Derivation

Batch construction. Batch size $B = 64$; each sequence has length (context size) $T = 512$; after the embedding layer every token is represented in $d_{\text{model}} = 768$ dimensions. Hence a batch of hidden states has shape

$$(B, T, C) = (64, 512, 768).$$

Loss. We minimize the (summed) cross-entropy between the true next-token distribution (one-hot y_i over the vocabulary) and the model’s predicted distribution $\hat{y}_i$, summed over the training corpus $\mathcal{X}$ and over positions $i = 1, \dots, T$. The slide writes

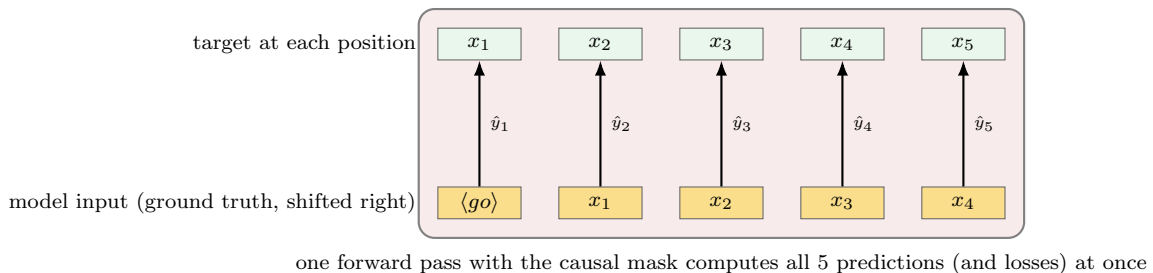
$$\mathcal{L} = - \sum_{x \in \mathcal{V}: X \in \mathcal{X}} \sum_{i=1}^T y_i \log(\hat{y}_i).$$

Unpacking the notation: X ranges over training sequences in the corpus $\mathcal{X}$; at each position i , $\hat{y}_i \in \mathbb{R}^{|\mathcal{V}|}$ is the predicted distribution and $y_i \in \{0, 1\}^{|\mathcal{V}|}$ is the one-hot vector of the true next token. The “ $x \in \mathcal{V}$ ” part is the sum over vocabulary entries inside the cross-entropy. Written out,

$$\mathcal{L} = - \sum_{X \in \mathcal{X}} \sum_{i=1}^T \sum_{v \in \mathcal{V}} y_{i,v} \log \hat{y}_{i,v} = - \sum_{X \in \mathcal{X}} \sum_{i=1}^T \log \hat{y}_{i, x_i},$$

because the one-hot y_i keeps only the log-probability of the correct token x_i . This is exactly the negative of the log-likelihood $\sum_i \log P(x_i | x_{<i})$ from Module 2: minimizing cross-entropy = maximizing likelihood.

Optimizer. Adam, with a cosine learning-rate schedule.



Intuition

Teacher forcing. During training we feed the model the *ground-truth* previous tokens $x_1, \dots, x_{i-1}$ (from the real training sequence) as the “history” when predicting x_i – rather than feeding back the model’s own (possibly wrong) predictions. Combined with the causal mask, this lets *all* T positions of a sequence be trained **in parallel** in a single forward pass, which is both fast and numerically stable (errors at one position do not compound into the next during training).

The trade-off is **exposure bias**: at inference time there is no ground truth to feed back, so the model must condition on its *own* previous outputs (Section 3.3.4). If the model makes a mistake

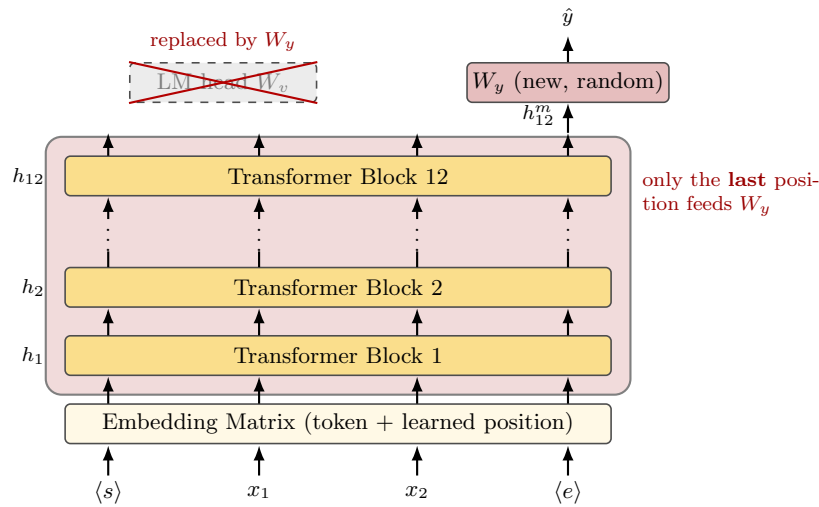
early in generation, that mistake becomes part of the “history” for every subsequent token – a mismatch between the training-time and inference-time input distributions that teacher forcing does not directly address.

3.2 Fine-tuning: Adapting GPT to Downstream Tasks

Key Idea

Fine-tuning re-purposes the pre-trained weights for a supervised task while changing the architecture as little as possible:

1. **Initialize** all transformer weights from the pre-trained (language-modelling) checkpoint.
2. **Modify the input:** wrap/format the raw tokens with task-specific special tokens – a start token $\langle s \rangle$, an end token $\langle e \rangle$, and (for multi-segment inputs) a delimiter $\langle \$ \rangle$.
3. **Modify the output:** *discard* the pre-training LM head W_v and attach a new, randomly-initialized task head W_y .



Derivation

Why only the last position? Because attention is *causal*, position i ’s representation has only ever attended to tokens $1, \dots, i$. The *last* time step m (i.e. the position of $\langle e \rangle$) is the *only* position that has attended to the **entire** input sequence. Hence its top-layer representation h_l^m (here $l = n = 12$) is the natural “sentence summary” to feed into a classification head:

$$\hat{y} = P(y \mid x_1, \dots, x_m) = \text{softmax}(W_y h_l^m), \quad h_l^m \in \mathbb{R}^{d_{\text{model}}}, \quad W_y \in \mathbb{R}^{c \times d_{\text{model}}}, \quad \hat{y} \in \mathbb{R}^c,$$

where c is the number of classes (e.g. $c = 2$ for positive/negative). The fine-tuning loss is the negative log-likelihood of the *correct* label over the labelled dataset $\mathcal{C}$:

$$\mathcal{L} = - \sum_{(x,y) \in \mathcal{C}} \log \hat{y}_y \quad (\text{the slide writes } \log(\hat{y}_i)).$$

W_y is *randomly initialized* (it did not exist during pre-training). If a sequence is shorter/longer than the context length $T = 512$, padding/truncation is applied.

Note / Gotcha

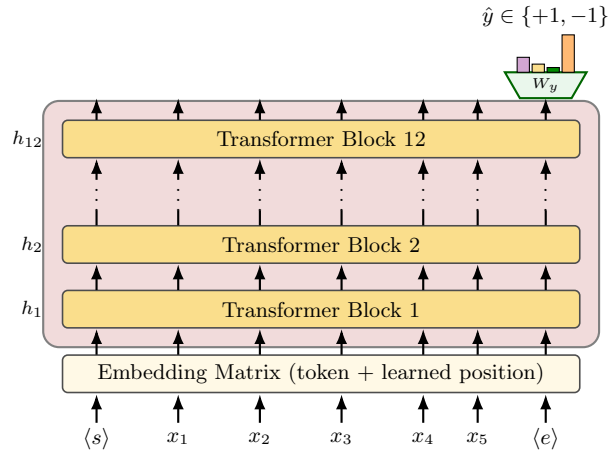
This differs from encoder-style models (e.g. BERT), which typically use a special [CLS] token placed at the *start* and pool bidirectionally. Because GPT is causal, a token at the start has seen *nothing* – so the aggregation point *must* be the last token instead.

Beyond the Slides

GPT-1 also kept the LM loss during fine-tuning. The original paper fine-tunes on $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{LM}}$ with $\lambda = 0.5$, where $\mathcal{L}_{\text{LM}}$ is the next-token loss on the same labelled text. This auxiliary term helped generalization and convergence. The only genuinely new parameters during fine-tuning are W_y and the embeddings of the added special tokens $\langle s \rangle, \langle e \rangle, \langle \$ \rangle$.

3.3 Case Studies: Formatting Inputs for Different Tasks

3.3.1 (i) Sentiment Analysis – Single-Sequence Classification



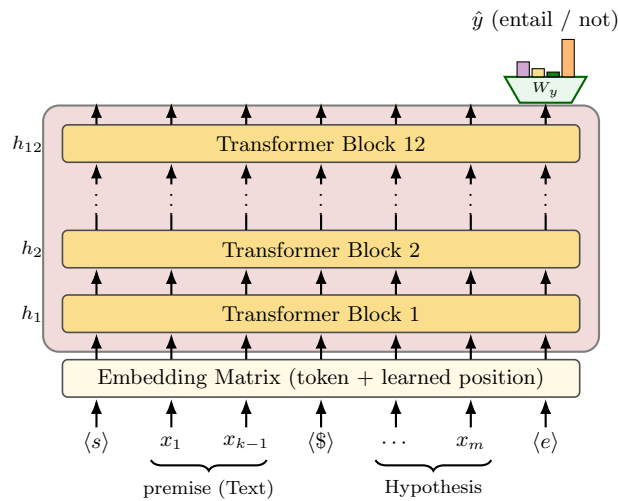
Worked Example

Text: “Wow, India has now reached the moon” $\rightarrow$ **Sentiment:** Positive ($\hat{y} = +1$). Input formatted as $\langle s \rangle$ Wow , India has now reached the moon $\langle e \rangle$, and the head $W_y \in \mathbb{R}^{2 \times 768}$ maps h_{12}^m to two scores, softmaxed into $P(+1), P(-1)$.

3.3.2 (ii) Textual Entailment / Contradiction – Sentence-Pair Classification

Here the input is *two* segments – a premise (“Text”) and a “Hypothesis”. Since a decoder-only model has a single input stream (no separate encoder for each sentence), both segments must be concatenated into *one* sequence, separated by a **delimiter token** $\langle \$ \rangle$:

$$\langle s \rangle \ x_1, \dots, x_{k-1} \ \langle \$ \rangle \ \dots, x_m \ \langle e \rangle.$$

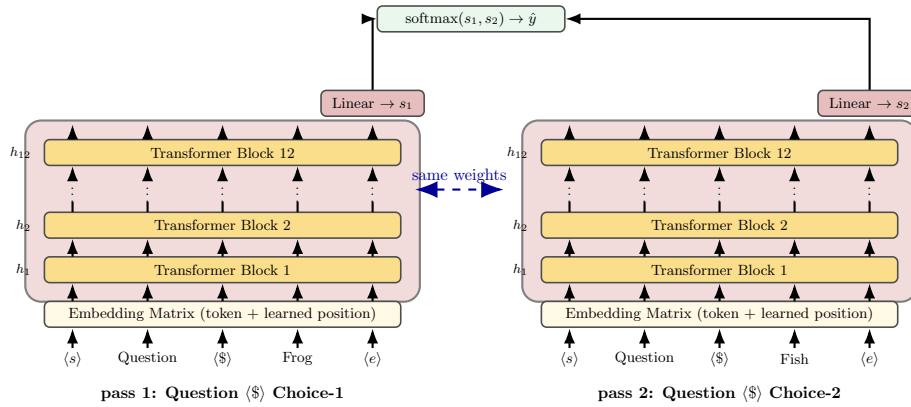


Worked Example

Text: “A soccer game with multiple males playing.” **Hypothesis:** “Some men are playing a sport.”
Entailment: True.

Intuition

Without $\langle \$ \rangle$, the model would see one run-on sequence of tokens with no signal about *where the premise ends and the hypothesis begins*. The delimiter is a learned embedding that the self-attention layers can key on to structurally separate the two segments, functionally similar to BERT’s segment/sentence-pair embeddings but implemented as an explicit vocabulary token instead of an added embedding.

3.3.3 (iii) Multiple Choice – Shared Encoder, Per-choice Heads**Key Idea**

For a question with k choices, construct k separate input sequences – *Question $\langle \$ \rangle$ Choice- i* , $i = 1, \dots, k$ – and run each through the **same** transformer (weights shared across all k passes). Each pass produces its own last-position representation $h_l^{m_i}$, which is scored by its own linear layer *Linear- i* to produce a scalar s_i . The k scores are then jointly normalized:

$$\hat{y} = \text{softmax}(s_1, \dots, s_k), \quad s_i = w_y^\top h_l^{m_i} \in \mathbb{R}, \quad w_y \in \mathbb{R}^{d_{\text{model}}}.$$

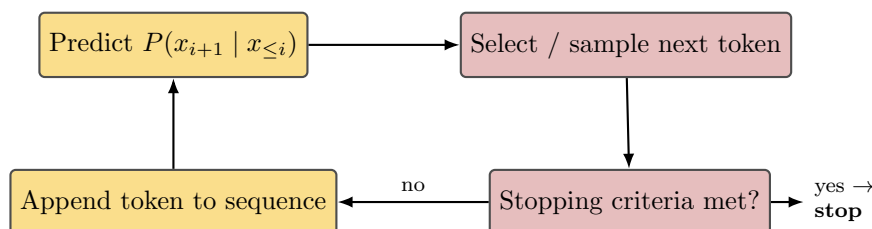
The slide draws “Linear-1” and “Linear-2” as separate boxes, but they are the same linear layer applied to each pass (as in the original GPT-1). Sharing it is what lets one model answer questions with any number of choices: the softmax, not the architecture, decides how many options compete.

Worked Example

Question: “Which of the following animals is an amphibian?” **Choices:** Frog, Fish. Run *Question $\langle \$ \rangle$ Frog* through the model to get score s_{Frog} , and *Question $\langle \$ \rangle$ Fish* to get s_{Fish} . Softmax-normalizing $(s_{\text{Frog}}, s_{\text{Fish}})$ and taking the arg max should select **Frog**.

3.3.4 (iv) Text Generation – Autoregressive Decoding

At inference, the pre-trained LM head is used directly (no fine-tuning needed for pure generation): feed a **prompt**, apply the causal mask, and repeatedly (1) predict the next token’s distribution, (2) select/sample a token, (3) append it to the input, and (4) repeat.



Worked Example

Prompt: “I like” (fed as $\langle s \rangle$, I, like) with the slide’s mask

$$M = \begin{bmatrix} 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \end{bmatrix}$$

and running the model autoregressively, with **stopping criteria** = max length 5 *or* generation of $\langle e \rangle$, we get **Output:** “I like to think that”.

Reading the slide’s mask: the first three columns are the prompt tokens, which are known, so they are unmasked; the last two columns are slots that have not been generated yet, so they are masked; row 4 is the first generated position, which may also see itself. In a standard GPT implementation the prompt positions keep the ordinary lower-triangular causal pattern among themselves too (exactly as in training), and each newly generated token simply adds one row and one column.

Intuition

Two natural questions arise: does the model always produce the *same* continuation for the same prompt (deterministic, e.g. *greedy* arg-max decoding), or should/can it be *creative* (stochastic sampling)? This tension motivates a wishlist of properties we want any decoding strategy to satisfy.

3.4 Wishlist for Text Generation

W1. Discourage degenerate text. Pure greedy/low-diversity decoding tends to produce:

- *Repetition loops*: “I like to think that I like to think ...”
- *Incoherent rambling*: “I like to think that reference know how to think best selling book”

W2. Encourage creativity/diversity. The *same* prompt should be able to produce different, equally plausible continuations across runs, e.g. for the prompt “I like”:

- “I like to read a book”
- “I like to buy a hot beverage”
- “I like a person who cares about others”

W3. Accelerate token generation. Autoregressive decoding is inherently **sequential** – token $i + 1$ cannot be produced until token i is available – so generation speed matters and is a first-class design concern (this motivates the decoding-strategy and inference-optimization techniques that build directly on top of the architecture covered here).

Beyond the Slides

The KV cache: why causal masking makes generation cheaper. A naive generator re-runs the whole model on the entire sequence for every new token, recomputing keys and values for positions it has already processed. But look at the mask: position i never attends to anything after it, so its keys $K[i]$ and values $V[i]$ in every layer are *fixed forever* once computed. A generator can therefore store (cache) them and, for each new token, compute only that one token’s q, k, v , attend over the cached keys/values, and append its own k, v to the cache.

- Work per new token drops from re-processing all t tokens to processing 1 token (attention still

reads all t cached entries).

- Memory cost: $2 \times n \times T \times d_{\text{model}}$ numbers per sequence (keys and values, every layer). For GPT-1 at full context: $2 \times 12 \times 512 \times 768 = 9,437,184$ numbers ≈ 37.7 MB in fp32 (18.9 MB in fp16). For modern long-context models this cache, not the weights, is often what limits how many requests fit on a GPU.

This is a direct consequence of Module 2's causal mask: a bidirectional encoder could not cache this way, because every new token would change every earlier position's representation.

Module Summary

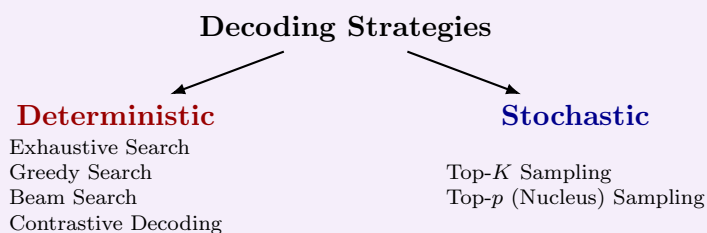
- Pre-training minimizes a summed cross-entropy CLM loss over large batches using teacher forcing, trading exposure bias for fast, parallel, stable training.
- Fine-tuning keeps the transformer backbone, swaps the LM head for a task head W_y , and reads off the representation at the *last* token position (the only position with full-sequence causal context).
- Different tasks are reduced to the *same* architecture purely by clever **input formatting**: special tokens $\langle s \rangle, \langle e \rangle$, a segment delimiter $\langle \$ \rangle$, and (for multiple choice) running k shared-weight passes with per-choice heads normalized jointly by softmax.
- Generation reuses the pre-trained LM head autoregressively; a good decoding strategy must balance fidelity/coherence, diversity, and speed.

Module 4 Decoding Strategies

Module 3 ended with the observation that generation is autoregressive: at each step the model outputs a full probability distribution $P(x_i | x_{<i})$ over the vocabulary, and *something* must turn that distribution into an actual next token. **Decoding** (a.k.a. *search*) is precisely this choice of procedure. It is applied identically regardless of which architecture produced the distributions (GPT, an encoder–decoder model, ...), which is why it is treated as its own module.

Key Idea

Decoding strategies split into two families:



Deterministic strategies always output the same sequence for the same input/model (they search for a single “best” sequence). **Stochastic** strategies deliberately introduce randomness, sampling from (a restricted, renormalized version of) the model’s own predicted distribution, trading some fidelity for diversity/creativity.

4.1 Exhaustive Search

Intuition

The conceptually simplest (and least practical) strategy: the true joint probability of a length- T sequence is $P(x_1, \dots, x_T) = \prod_{i=1}^T P(x_i | x_{<i})$ (Eq. (1)). So, in principle, enumerate *every* possible sequence of length T over the vocabulary $\mathcal{V}$, compute each one’s probability with the model, and output the single sequence with the highest probability. This is the *definition* of the best sequence; every other deterministic method is an approximation of it.

Worked Example

Generate 5 tokens with $\mathcal{V} = \{\text{cold, coffee, I, like, water, } \langle \text{stop} \rangle\}$. The search space holds $|\mathcal{V}|^5 = 6^5 = 7,776$ sequences. A few of them, with the probabilities shown on the slide:

Sequence	$p(\cdot)$	
“I like cold water”	0.10	
“I like cold coffee”	0.15	← highest of all 7,776, so this is the output
“I like coffee”	0.12	
“coffee like cold coffee”	0.10	
“I like I like”	0.001	
“coffee coffee coffee coffee”	0.0001	

(Shorter strings are padded with $\langle \text{stop} \rangle$ to length 5. The slide marks the output with an asterisk: *assuming* it is the highest among all $|\mathcal{V}|^5$ sequences, which is exactly what exhaustive search checks.)

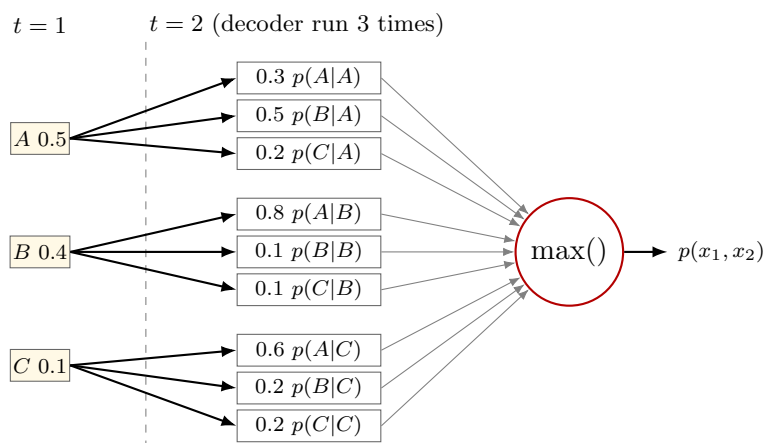
Derivation

Counting the work. At step 1 the decoder runs once and outputs $|\mathcal{V}|$ probabilities. To continue *every* length-1 prefix, step 2 must run the decoder once *per prefix*, conditioned on it: $|\mathcal{V}|$ runs. Step

3 has $|\mathcal{V}|^2$ prefixes, so $|\mathcal{V}|^2$ runs, and so on:

decoder runs at step $t = |\mathcal{V}|^{t-1}$,
 candidate sequences after step $t = |\mathcal{V}|^t$,
 sequences to compare at the end = $|\mathcal{V}|^T$.

Slide check ($|\mathcal{V}| = 3$): step 1 runs once, step 2 runs 3 times, step 3 runs 9 times. With $|\mathcal{V}| = 40,000$, step 2 alone already needs 40,000 decoder runs in parallel, and the count multiplies by 40,000 at every further step. For a GPT-1-sized vocabulary, $|\mathcal{V}|^T$ passes the number of atoms in the observable universe ($\sim 10^{80}$) at about $T = 18$ tokens ($40,478^{18} \approx 10^{83}$).



Worked Example

Finishing the slide's toy example. Multiply each path: $p(x_1, x_2) = p(x_1) p(x_2 | x_1)$.

path	product	=	path	product	=
AA	0.5×0.3	0.15	BB	0.4×0.1	0.04
AB	0.5×0.5	0.25	BC	0.4×0.1	0.04
AC	0.5×0.2	0.10	CA	0.1×0.6	0.06
BA	0.4×0.8	0.32	CB	0.1×0.2	0.02
			CC	0.1×0.2	0.02

The nine products sum to 1 (a sanity check), and the winner is **BA** with 0.32. Now notice what greedy search (next section) would do on the same numbers: pick *A* at $t = 1$ (0.5 is largest), then *B* (0.5 is largest given *A*), giving $AB = 0.25 < 0.32$. The token that looked best at step 1 was not the start of the best sequence.

4.2 Greedy Search

Key Idea

At *every* time step, output the single token with the highest conditional probability – i.e. take a local arg max instead of a global search:

$$\hat{x}_i = \arg \max_{x \in \mathcal{V}} P(x | \hat{x}_{<i}).$$

This needs only one decoder run per position, T runs in total, instead of exponentially many.

Worked Example

The slide’s greedy run. Each column is the model’s distribution at that step (it sums to 1); the boxed entry is the arg max that greedy picks.

token	time step				
	1	2	3	4	5
cold	0.1	0.1	0.45	0.15	0.1
⟨stop⟩	0.15	0.15	0.05	0.3	0.5
coffee	0.25	0.25	0.1	0.35	0.2
I	0.4	0.05	0.05	0.01	0.1
like	0.05	0.35	0.15	0.09	0.05
water	0.05	0.1	0.2	0.1	0.05
picked	I	like	cold	coffee	⟨stop⟩

$$p(\text{I like cold coffee } \langle \text{stop} \rangle) = 0.4 \times 0.35 \times 0.45 \times 0.35 \times 0.5 = \mathbf{0.011}.$$

Note / Gotcha

Greedy is provably suboptimal. Suppose instead we pick the *second*-best token at step 1 (“coffee”, 0.25). The model is then conditioned on a different prefix, so *columns 2–5 change completely*:

token	1	2	3	4	5
cold	0.1	0.15	0.65	0.04	0.1
⟨stop⟩	0.15	0.1	0.05	0.01	0.5
coffee	0.25	0.05	0.05	0.1	0.2
I	0.4	0.05	0.05	0.03	0.1
like	0.05	0.55	0.1	0.02	0.05
water	0.05	0.1	0.1	0.8	0.05
picked	coffee	like	cold	water	⟨stop⟩

$$p(\text{coffee like cold water } \langle \text{stop} \rangle) = 0.25 \times 0.55 \times 0.65 \times 0.8 \times 0.5 = \mathbf{0.0358} > 0.011.$$

Greedy search is **myopic**: a locally best choice at step i can lead into a region where every later conditional is mediocre, while a slightly worse choice can unlock a path of confident later steps. Because probabilities multiply, several strong later factors easily outweigh one weaker early factor.

Exam Trap

Do not read a greedy table as one fixed matrix and pick the best path through it. Only column 1 is fixed; every later column is $P(\cdot \mid \text{prefix})$ and is *recomputed* whenever the prefix changes. That is why the two tables above share column 1 and nothing else.

Intuition

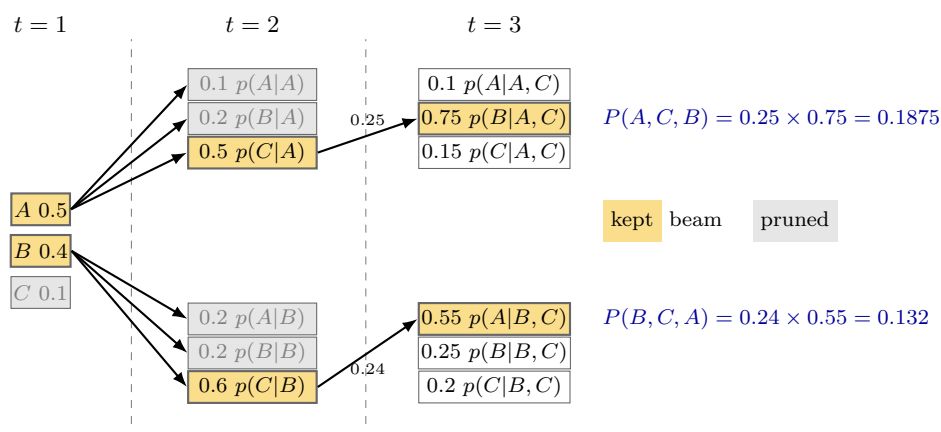
Beyond sub-optimality, greedy search has two more practical problems: (1) it is **fully deterministic** – the same prompt always produces exactly the same continuation (start from “I” and you always get “I like cold coffee”); and (2) it is prone to **repetition loops**, because once a phrase is highly probable in its own context, repeating it often stays the arg max.

4.3 Beam Search

Key Idea

Beam search is the middle ground between exhaustive search (track *all* $|\mathcal{V}|^t$ partial sequences) and greedy search (track only the single best one). Fix a **beam size** k : at every step keep only the top- k highest-probability *partial sequences* (“beams”), expand each by all $|\mathcal{V}|$ possible next tokens ($k \times |\mathcal{V}|$ candidates), and prune back to the top- k by cumulative probability.

work per step = k decoder runs, $k \times |\mathcal{V}|$ candidate scores (linear in k).



Worked Example

$k = 2$, toy vocabulary $\{A, B, C\}$ (the slide’s numbers).

Step 1. $p(A) = 0.5$, $p(B) = 0.4$, $p(C) = 0.1$: keep beams A and B ; C is pruned.

Step 2. Expand each beam by all 3 tokens: $2 \times 3 = 6$ candidates, scored by $p(w_1)p(w_2 | w_1)$:

$$\begin{aligned}
 p(A)p(A|A) &= 0.5 \times 0.1 = 0.05 & p(B)p(A|B) &= 0.4 \times 0.2 = 0.08 \\
 p(A)p(B|A) &= 0.5 \times 0.2 = 0.10 & p(B)p(B|B) &= 0.4 \times 0.2 = 0.08 \\
 p(A)p(C|A) &= 0.5 \times 0.5 = \mathbf{0.25} & p(B)p(C|B) &= 0.4 \times 0.6 = \mathbf{0.24}
 \end{aligned}$$

Keep “AC” and “BC”. Note that both survivors end in C , a token that step 1 had *pruned* as a first word. Beam search keeps sequences, not tokens.

Step 3. Expand again ($2 \times 3 = 6$ candidates):

$$\begin{aligned}
 ACA : 0.25 \times 0.1 &= 0.025 & BCA : 0.24 \times 0.55 &= \mathbf{0.132} \\
 ACB : 0.25 \times 0.75 &= \mathbf{0.1875} & BCB : 0.24 \times 0.25 &= 0.060 \\
 ACC : 0.25 \times 0.15 &= 0.0375 & BCC : 0.24 \times 0.2 &= 0.048
 \end{aligned}$$

The final beams are ACB (0.1875; the slide rounds to 0.18) and BCA ($0.132 \approx 0.13$). The output is the higher one, **ACB**. With $k = 1$ the procedure would keep only one beam at every step, which is exactly greedy search.

Derivation

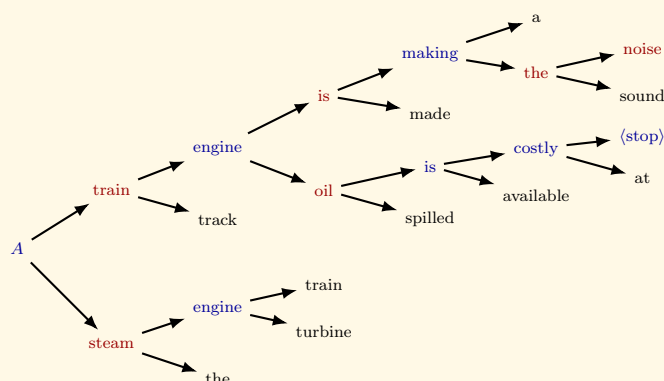
Termination and length normalization. After T steps we have k complete candidate sequences; output the one with the highest *overall* probability. But since every factor $P(x_i | x_{<i}) \leq 1$, a longer sequence is a product of *more* such factors and is therefore mechanically penalized relative to a shorter one, *regardless of quality*. The standard fix is **length normalization**: divide the (log-)probability by the sequence length before comparing,

$$\text{score}(x_1, \dots, x_T) = \frac{1}{T} \sum_{i=1}^T \log P(x_i | x_{<i}),$$

so sequences of different lengths are compared fairly.

Worked Example

Backtracking example. Expanding and pruning with some beam width from a start symbol $\langle A \rangle$ over several steps might produce a search graph like:



Backtracking the highest-scoring, length-normalized path(s) recovers, e.g., “A train engine is making the noise” and “A train engine oil is costly” – two grammatically complete, plausible sentences the beam search explored in parallel.

Note / Gotcha

- Beam width $k = 1$ reduces exactly to greedy search; larger k is a strictly better approximation of exhaustive search, at a roughly linear (in k) increase in compute and latency.
- Both greedy and beam search remain **deterministic** and **prone to degenerate, repetitive text** (this is a separate failure mode from sub-optimality – see below).
- Latency: greedy < beam search (beam tracks and scores $k \times$ more candidates at every step).
- Neither greedy nor beam search can produce a *variety* of outputs for the same input – both are still just deterministic search procedures, only searching a larger/smaller space. This makes beam search excellent for tasks with one “correct-ish” target output (**machine translation**, **summarization**), but poor for open-ended **creative** generation.

Intuition

Why beam search text feels bland: surprise is a product of randomness. We are surprised by an outcome precisely when it was *uncertain* – $\text{Surprise} \approx \text{uncertainty}$. Formally, the “surprise”/self-information of an event x with probability $p(x)$ is $-\log p(x)$, and its expectation over the distribution is exactly the (Shannon) **entropy**. Comparing the token-level probability the model assigned to what actually got produced, *at every time step*, for real human-written text versus beam-search-generated text reveals a striking pattern:



Beam search keeps the per-step probability of its own chosen token pinned near 1 almost always (*low variance* – it always takes the safest, most predictable option). **Human-written** text has *high variance*, frequently choosing far less predictable (lower-probability, more “surprising”) words. This is exactly why purely deterministic search feels generic/bland: it systematically avoids the very unpredictability that makes text feel natural and creative. This observation – that intentionally

giving weight to *other*, less-than-maximal-probability tokens is necessary for natural, varied text – directly motivates **stochastic (sampling-based) decoding**.

4.4 Sampling Strategies

4.4.1 Top- K Sampling

Key Idea

At every time step, restrict attention to the K highest-probability tokens, **renormalize** their probabilities so they sum to 1, and then *sample* the next token from this restricted distribution instead of taking the arg max. Because sampling is random, running the same model on the same prompt twice can produce two different sequences. $K = 1$ is greedy search; $K = |\mathcal{V}|$ is plain (unrestricted) sampling.

Derivation

If the raw (full-vocabulary) probabilities of the top- K tokens are $p_1, \dots, p_K$, the renormalized sampling distribution is

$$\tilde{p}_j = \frac{p_j}{\sum_{j'=1}^K p_{j'}}, \quad j = 1, \dots, K,$$

and every token outside the top K gets probability exactly 0.

Worked Example

The slide’s two runs with $K = 2$. At step 1 the top two tokens are I (0.4) and coffee (0.25). Renormalized:

$$\tilde{p}(\text{I}) = \frac{0.4}{0.4 + 0.25} = 0.615, \quad \tilde{p}(\text{coffee}) = \frac{0.25}{0.65} = 0.385.$$

Run 1 draws I, then draws ⟨stop⟩ from the next top-2, giving “I ⟨stop⟩”. *Run 2* goes like this:

step	top-2 candidates (raw)	renormalized	drawn	$\tilde{p}$ of draw
1	I 0.4, coffee 0.25	0.615, 0.385	coffee	0.385
2	like 0.55, ⟨stop⟩ 0.15	0.786, 0.214	like	0.786
3	cold 0.65, water 0.11	0.855, 0.145	cold	0.855
4	water 0.8, coffee 0.1	0.889, 0.111	coffee	0.111
5	⟨stop⟩ 0.5, coffee 0.2	0.714, 0.286	⟨stop⟩	0.714

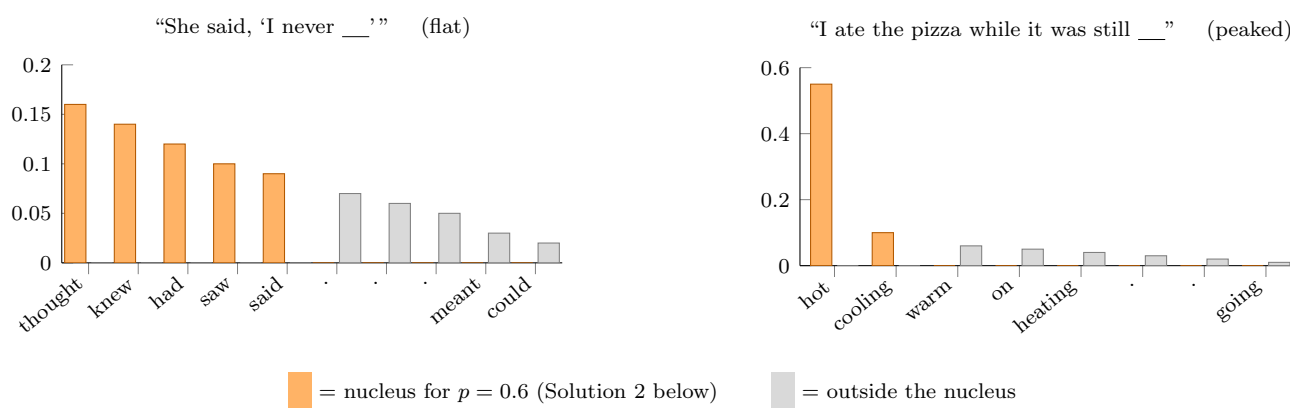
Result: “coffee like cold coffee ⟨stop⟩”. The sampler produced this exact sequence with probability $0.385 \times 0.786 \times 0.855 \times 0.111 \times 0.714 \approx 0.021$, about 1 run in 50. Step 4 is the interesting one: the sampler picked the *second* choice, **coffee**, which it only does about 11% of the time. That is the “surprise” that beam search never produces: a lower-probability but still sensible token.

Intuition

How does random sampling help? The slide’s answer is the beam-vs-human plot above: human text regularly picks tokens that are *not* the most probable, while beam search almost never does. Top- K sampling gives those other highly probable tokens a chance, in proportion to their probability, which produces variety across runs. Restricting to the top K keeps it from ever picking the long tail of nonsense tokens.

4.4.2 Choosing K is Hard: the Flat vs. Peaked Problem

The next-token distribution changes shape from step to step. The slide shows two contexts (bar heights below are illustrative values chosen to match the slide’s picture and its $p = 0.6$ example):



Note / Gotcha

Fix $K = 5$ and see what happens in each context:

- **Flat** (left): many words after “I never” are about equally plausible. $K = 5$ cuts off **meant**, **could** and the others even though they are nearly as likely as **said**. The model misses valid sentences, so it is *less creative* than it could be.
- **Peaked** (right): the model is confident that the word is **hot**. The same $K = 5$ still lets in **on** and **heating**, tokens the model considers unlikely, and sampling them produces sentences like “while it was still on” or “still heating” that do not fit. The model can produce *meaningless* output.

A fixed K is too small for flat distributions and too large for peaked ones. The number of candidates has to follow the model’s confidence at each step.

4.4.3 Solution 1: Temperature Sampling

Derivation

Given the pre-softmax logits $u_1, \dots, u_{|\mathcal{V}|}$ and a **temperature** $\mathcal{T} > 0$ (the slide writes T), define the temperature-scaled softmax:

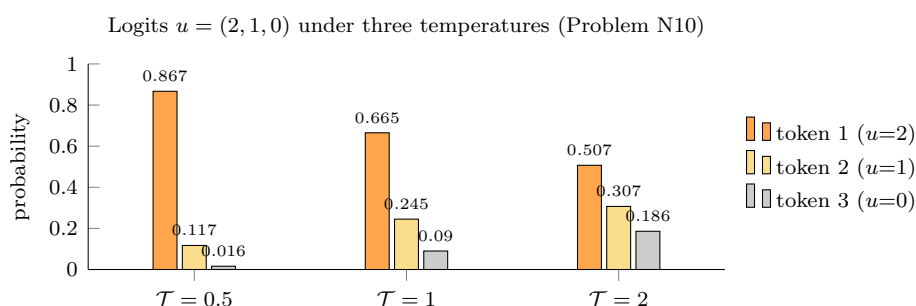
$$P(x = u_l \mid x_{1:i-1}) = \frac{\exp(u_l/\mathcal{T})}{\sum_{l'} \exp(u_{l'}/\mathcal{T})}.$$

Dividing numerator and denominator by $\exp(u_l/\mathcal{T})$ gives a form that makes the limits easy to see:

$$P(x = u_l \mid \cdot) = \frac{1}{\sum_{l'} \exp((u_{l'} - u_l)/\mathcal{T})}.$$

- $\mathcal{T} = 1$: the ordinary softmax.
- $\mathcal{T} \rightarrow 0^+$: if u_l is the (unique) largest logit, every other exponent $(u_{l'} - u_l)/\mathcal{T} \rightarrow -\infty$, so the sum $\rightarrow 1$ and $P \rightarrow 1$; every other token’s sum contains a term $\rightarrow +\infty$, so its $P \rightarrow 0$. The distribution becomes one-hot on the arg max: **temperature 0 is greedy search**.
- $\mathcal{T} \rightarrow \infty$: every exponent $\rightarrow 0$, each term $\rightarrow 1$, the sum $\rightarrow |\mathcal{V}|$ and $P \rightarrow 1/|\mathcal{V}|$: **uniform sampling**.

Temperature never changes the *order* of the tokens, only how far apart their probabilities are.



Intuition

Low temperature $\Rightarrow$ skewed distribution $\Rightarrow$ less creativity (closer to greedy, more repetition).

High temperature $\Rightarrow$ flatter distribution $\Rightarrow$ more creativity, but also more risk of incoherence.

How it relates to the K problem: temperature does not choose *which* tokens are eligible, it reshapes the whole distribution, so on its own it cannot make a flat step more selective and a peaked step less selective at the same time. It is usually combined with top- K or top- p : scale the logits first, then restrict and sample.

4.4.4 Solution 2: Top- p (Nucleus) Sampling**Key Idea**

Instead of fixing the *number* of tokens K , fix the *probability mass* $p \in (0, 1)$:

1. Sort the probabilities in descending order.
2. Add them up starting from the top token.
3. Stop as soon as the running sum exceeds p . The tokens added so far are the **nucleus**.
4. Renormalize within the nucleus and sample a token from it.

It is top- K with K chosen dynamically, per step.

Worked Example

$p = 0.6$ on the two contexts above.

	running sum	nucleus
flat (“I never”)	0.16, 0.30, 0.42, 0.52, 0.61 > 0.6	{thought, knew, had, saw, said}: 5 tokens
peaked (“still”)	0.55, 0.65 > 0.6	{hot, cooling}: 2 tokens

These are exactly the sets on the slide. On the peaked step, sampling inside the nucleus gives $\tilde{p}(\text{hot}) = 0.55/0.65 = 0.846$ and $\tilde{p}(\text{cooling}) = 0.154$; **on** and **heating** can no longer be drawn. The nucleus **grows** when the model is uncertain and **shrinks** when it is confident, which is exactly what the fixed- K problem required.

Beyond the Slides

How these combine in practice. Production LLM samplers rarely use one method alone. A common recipe: (1) divide the logits by a temperature, (2) keep the top- p nucleus (often with a hard top- K cap as well), (3) renormalize and sample. Two other widely used tools that the deck does not cover: **repetition / frequency penalties**, which lower the logits of tokens that have already appeared, directly targeting the repetition loops of W1; and **contrastive decoding**, which appears in the slide’s taxonomy but is not explained there. Roughly, it scores candidate tokens by how much more likely a strong model finds them than a weaker model does, favoring tokens that are both plausible and distinctive rather than merely generic.

Module Summary

Strategy	Idea	Decoder runs at step t	Random?	Main weakness / use
Exhaustive	score every sequence, keep the max	$ \mathcal{V} ^{t-1}$	no	exact, but exponential
Greedy	arg max at each step	1	no	myopic; repeats; no variety
Beam (k)	keep top- k partial sequences	k	no	still deterministic and bland; good for translation, summarization
Top- K	sample from renormalized top K	1	yes	one K cannot fit flat and peaked steps
Top- p	sample from smallest prefix with mass $> p$	1	yes	adapts candidate count to confidence
Temperature	rescale logits by $1/T$	1	(modifier)	sharpens or flattens; combined with the above

Wishlist check: **W1** (avoid degeneration) and **W2** (creativity) are served by sampling (top- K /top- p , temperature); **W3** (speed) favors greedy and sampling (one run per step) over beam search (k runs per step).

References (as cited in the deck)

- [1] Radford et al., *Improving Language Understanding by Generative Pre-Training* (GPT-1) – <https://openai.com/research/language-unsupervised>
- [2] *Beam Search Strategies for Neural Machine Translation*
- [3] Holtzman et al., *The Curious Case of Neural Text Degeneration* (introduces nucleus/top- p sampling)
- [4] Maxime Labonne, *Decoding Strategies* (blog post)

Module 5 One-Page Revision Sheet

GPT-1 numbers

Corpus	BookCorpus: 7,000 books, 74M sentences, $\approx 1\text{B}$ words, 16 genres
Tokenizer	BPE, $ \mathcal{V} = 40,478$
Blocks n	12 (decoder-only)
Context T	512
d_{model}	768
Heads H, d_k	12, 64
d_{ff}	3072 (4×768)
Activation	GELU
Positions	learned 512×768
Batch	$64 \Rightarrow (64, 512, 768)$
Optimizer	Adam + cosine LR
Params	$\approx 117\text{M}$ (116,461,056)

Formulas

$P(x_{1:T}) = \prod_i P(x_i | x_{<i})$
 $P(x_i) = \text{softmax}(h_n[i] W_v)$
 $\mathcal{L}_{\text{pre}} = - \sum_X \sum_i \log \hat{y}_{i,x_i}$
 $A = \text{softmax}(QK^\top / \sqrt{d_k} + M)$
 $\hat{y} = \text{softmax}(W_y h_{12}^m)$ (last token)
 Params/block $\approx 12d^2$ ($4d^2$ attn, $8d^2$ FFN)
 Top- K : $\tilde{p}_j = p_j / \sum_{j' \leq K} p_{j'}$
 Temp.: $\text{softmax}(u/\mathcal{T})$; $\mathcal{T} \rightarrow 0 = \text{greedy}$
 Top- p : smallest top set with mass $> p$
 Beam: k runs and $k|\mathcal{V}|$ scores per step

Exam Trap

1. A GPT block has *no* cross-attention (no encoder); it still needs *masked* self-attention.
2. The mask is added *before* softmax and uses $-\infty$, not 0. Post-softmax, masked weights are exactly 0.
3. $d_k = d_{\text{model}}/H = 64$, so W_Q, W_K, W_V per head are 768×64 , not 768×768 .
4. Classification reads the *last* position ($\langle e \rangle$), not the first; the first token has seen nothing.
5. Entailment needs the delimiter $\langle \$ \rangle$ between premise and hypothesis.
6. Multiple choice: one pass per choice, shared weights, *one* softmax over all choice scores.
7. GPT-1's positional embeddings are *learned*; the original Transformer's were fixed sinusoids.
8. The FFN holds about twice the parameters of attention per block; the LM head adds none (tied to embeddings).
9. Greedy $\neq$ most probable sequence. Every column of a greedy table after the first depends on the prefix.
10. Beam search with $k = 1$ is greedy. Beam search is still deterministic; a bigger k does not add variety.
11. Low temperature = sharper = less creative; high temperature = flatter = more creative.
12. Top- p adapts the candidate count: many tokens on flat steps, few on peaked steps.
13. The slide's FFN figure says 3042; the correct width is 3072.

Module 6 Practice: Test Your Knowledge

6.1 Numerical Problems (with Full Worked Solutions)

Problem N1 – Parameter counting for a smaller GPT

A “GPT-mini” has: vocabulary $|\mathcal{V}| = 30,000$, embedding dimension $d_{\text{model}} = 512$, context length $T = 256$, $n = 6$ transformer blocks, $H = 8$ attention heads, and FFN hidden size $d_{\text{ff}} = 2048$. Compute the total parameter count (embeddings + attention + FFN), showing every intermediate dimension.

Solution

Embeddings.

Token: $30,000 \times 512 = 15,360,000$, Positional: $256 \times 512 = 131,072$.

Embedding total = $15,360,000 + 131,072 = 15,491,072 \approx 15.49\text{M}$.

Attention (per block). $d_k = d_{\text{model}}/H = 512/8 = 64$. Per head, Q, K, V : $3 \times (512 \times 64) = 98,304$. For 8 heads: $8 \times 98,304 = 786,432$. Output projection $W_O \in \mathbb{R}^{512 \times 512}$: $512 \times 512 = 262,144$.

Attention per block = $786,432 + 262,144 = 1,048,576 \approx 1.05\text{M}$.

For 6 blocks: $6 \times 1,048,576 = 6,291,456 \approx 6.29\text{M}$.

FFN (per block). $W_1 \in \mathbb{R}^{512 \times 2048}$, $W_2 \in \mathbb{R}^{2048 \times 512}$, biases $2048 + 512$:

$2 \times (512 \times 2048) + 2048 + 512 = 2,097,152 + 2,560 = 2,099,712 \approx 2.10\text{M}$.

For 6 blocks: $6 \times 2,099,712 = 12,598,272 \approx 12.60\text{M}$.

Grand total:

$15,491,072 + 6,291,456 + 12,598,272 = 34,380,800 \approx \mathbf{34.38}$ Million parameters.

Problem N2 – Writing out a causal mask

For a sequence of length $T = 6$, write the causal mask $M \in \mathbb{R}^{6 \times 6}$ (using $0/-\infty$), and state how many (query, key) pairs are *allowed* (non-masked) in total.

Solution

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Row i (0-indexed from 1) has exactly i allowed entries (positions $1, \dots, i$). Total allowed pairs = $1 + 2 + 3 + 4 + 5 + 6 = \frac{T(T+1)}{2} = \frac{6 \times 7}{2} = \mathbf{21}$ out of $T^2 = 36$ total entries – i.e. 58.3% of the full attention matrix is used; the remaining 41.7% is masked out.

Problem N3 – Numeric causal self-attention by hand

Consider $T = 3$ tokens with single-head $d_k = 2$ query/key/value vectors (already computed):

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}.$$

Compute the causally-masked attention output $O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$.

Solution

Step 1: raw scores $QK^\top$.

$$QK^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Step 2: scale by $1/\sqrt{d_k} = 1/\sqrt{2} \approx 0.7071$.

$$\frac{QK^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

Step 3: apply causal mask (row 1: only col 1 allowed; row 2: cols 1–2; row 3: all):

$$S = \begin{bmatrix} 0.7071 & -\infty & -\infty \\ 0 & 0.7071 & -\infty \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

Step 4: row-wise softmax. Row 1: only one unmasked entry $\Rightarrow A_1 = (1, 0, 0)$.

Row 2: $\text{softmax}(0, 0.7071)$: $e^0 = 1$, $e^{0.7071} = 2.0281$; sum = 3.0281 $\Rightarrow A_2 = (0.3303, 0.6697, 0)$.

Row 3: $\text{softmax}(0.7071, 0.7071, 1.4142)$: $e^{0.7071} = 2.0281$, $e^{0.7071} = 2.0281$, $e^{1.4142} = 4.1133$; sum = 8.1695 $\Rightarrow A_3 = (0.2482, 0.2482, 0.5035)$. **Step 5: weighted sum with V , $O = AV$.**

$$O_1 = 1 \cdot (1, 2) = (1, 2).$$

$$O_2 = 0.3303 \cdot (1, 2) + 0.6697 \cdot (3, 4) = (0.3303 + 2.0091, 0.6606 + 2.6788) = (2.339, 3.339).$$

$$O_3 = 0.2482(1, 2) + 0.2482(3, 4) + 0.5035(5, 6) = (0.2482 + 0.7446 + 2.5175, 0.4964 + 0.9928 + 3.021) \\ = (3.510, 4.510).$$

$$O \approx \begin{bmatrix} 1.000 & 2.000 \\ 2.339 & 3.339 \\ 3.510 & 4.510 \end{bmatrix}$$

Notice row 1's output is exactly V_1 (it can only attend to itself), consistent with the causal mask.

Problem N4 – FFN expansion cost

For GPT-1's FFN ($d_{\text{model}} = 768 \rightarrow d_{\text{ff}} = 3072 \rightarrow d_{\text{model}} = 768$), how many multiply-add operations (MACs) are needed for *one* token to pass through the FFN (ignore bias-add cost)? Express your answer in millions of MACs.

Solution

A matrix–vector product $W \in \mathbb{R}^{a \times b}$ applied to a length- a vector costs $a \times b$ MACs.

Layer 1 ($768 \rightarrow 3072$) : $768 \times 3072 = 2,359,296$ MACs.

Layer 2 ($3072 \rightarrow 768$) : $3072 \times 768 = 2,359,296$ MACs.

Total = $2 \times 2,359,296 = 4,718,592$ MACs $\approx$ **4.72** million MACs per token, per block.

(This matches the 4.7M *parameter* count derived in Section 2.5(d) – for a single dense layer applied once, #MACs = #weight-parameters, since each weight is used exactly once per token.)

Problem N5 – Cross-entropy loss and perplexity

A trained GPT model assigns the following probabilities to the *true* next tokens across a 4-token evaluation sequence: 0.5, 0.25, 0.8, 0.1. Compute (a) the total cross-entropy loss $\mathcal{L} = -\sum_i \log \hat{y}_i$ (natural log), and (b) the corresponding perplexity $\text{PPL} = \exp(\mathcal{L}/T)$.

Solution

(a) $\mathcal{L} = -[\ln 0.5 + \ln 0.25 + \ln 0.8 + \ln 0.1]$

$$= -[(-0.6931) + (-1.3863) + (-0.2231) + (-2.3026)] = -(-4.6051) = 4.6051.$$

(b) $T = 4$, so average loss per token = $4.6051/4 = 1.1513$.

$$\text{PPL} = e^{1.1513} \approx \mathbf{3.163}.$$

Interpretation: a perplexity of ≈ 3.16 means the model is, on average, as “confused” at each step as if it had to choose uniformly among ≈ 3.16 equally likely tokens – lower perplexity indicates a better-calibrated, more confident (and typically more accurate) language model.

Problem N6 – Multiple-choice softmax normalization

In the multiple-choice fine-tuning setup, a question has 4 choices whose per-choice linear heads output raw scores $s = (2.1, 0.4, -1.0, 1.8)$. Compute the softmax-normalized probabilities and identify the predicted choice.

Solution

$$e^{2.1} = 8.166, \quad e^{0.4} = 1.492, \quad e^{-1.0} = 0.368, \quad e^{1.8} = 6.050.$$

$$\text{Sum} = 8.166 + 1.492 + 0.368 + 6.050 = 16.076.$$

$$\hat{y} = \left(\frac{8.166}{16.076}, \frac{1.492}{16.076}, \frac{0.368}{16.076}, \frac{6.050}{16.076} \right) \approx (0.508, 0.093, 0.023, 0.376).$$

The largest probability is 0.508 for **Choice 1** – so the model predicts choice 1 (note choice 4, at 0.376, is a fairly close second, reflecting the raw scores 2.1 vs. 1.8).

Problem N7 – Cost of exhaustive search at scale

For GPT-1’s vocabulary ($|\mathcal{V}| = 40,478$) and a target length $T = 10$: (a) how many candidate sequences must exhaustive search compare, as a power of 10? (b) How many decoder runs does it need in total? (c) At an absurdly optimistic 10^{18} decoder runs per second, how long would it take?

Solution

(a) $\log_{10}(40,478) \approx 4.607$, so

$$|\mathcal{V}|^{10} = 10^{46.07} \approx 1.2 \times 10^{46} \text{ sequences.}$$

(b) Step t needs $|\mathcal{V}|^{t-1}$ runs (one per prefix of length $t-1$):

$$\sum_{t=1}^{10} |\mathcal{V}|^{t-1} = 1 + |\mathcal{V}| + \dots + |\mathcal{V}|^9 \approx |\mathcal{V}|^9 = 10^{41.46} \approx 2.9 \times 10^{41} \text{ runs.}$$

(c) $2.9 \times 10^{41} / 10^{18} = 2.9 \times 10^{23}$ seconds. Dividing by 3.15×10^7 seconds per year gives $\approx 9 \times 10^{15}$ years, roughly 650,000 times the age of the universe (1.4×10^{10} years), for a 10-token sentence.

Problem N8 – Beam search by hand ($k = 2$)

Vocabulary $\{X, Y, Z\}$. Step-1 probabilities: $p(X) = 0.6$, $p(Y) = 0.3$, $p(Z) = 0.1$. Step-2 conditionals: $p(X|X) = 0.1$, $p(Y|X) = 0.2$, $p(Z|X) = 0.7$; $p(X|Y) = 0.5$, $p(Y|Y) = 0.4$, $p(Z|Y) = 0.1$. Run beam search with $k = 2$ for two steps and report the surviving beams with their scores.

Solution

Step 1: keep top-2 by probability: beams $\{X:0.6, Y:0.3\}$ (Z pruned, $0.1 < 0.3$).

Step 2: expand each beam by all 3 tokens and score by cumulative product:

$$\begin{array}{ll} p(X)p(X|X) = 0.6 \times 0.1 = 0.06 & p(Y)p(X|Y) = 0.3 \times 0.5 = 0.15 \\ p(X)p(Y|X) = 0.6 \times 0.2 = 0.12 & p(Y)p(Y|Y) = 0.3 \times 0.4 = 0.12 \\ \boxed{p(X)p(Z|X) = 0.6 \times 0.7 = 0.42} & p(Y)p(Z|Y) = 0.3 \times 0.1 = 0.03 \end{array}$$

Sorting all 6 scores: $0.42 (XZ) > 0.15 (YX) > 0.12 (XY) = 0.12 (YY) > 0.06 (XX) > 0.03 (YZ)$. Keeping the top-2: surviving beams are “XZ” (score 0.42) and “YX” (score 0.15). Note that “XY” and “YY” tie at 0.12 but both lose out to the top two – a beam search with $k = 2$ discards them, whereas exhaustive search would have kept every one of these 6 candidates.

Problem N9 – Constructing the top- p nucleus

A sorted next-token distribution is $p = (0.32, 0.22, 0.15, 0.11, 0.08, 0.06, 0.04, 0.02)$ (already descending, sums to 1). For $p_{\text{thresh}} = 0.7$, determine the nucleus (which tokens are included) and the renormalized sampling probability of the *first* token in the nucleus.

Solution

Running cumulative sum: 0.32, 0.54, 0.69, 0.80, ... The sum first *exceeds* 0.7 once the 4th token is included ($0.80 > 0.7$; the first three alone only reach 0.69). So the nucleus is the first 4 tokens, with raw probabilities (0.32, 0.22, 0.15, 0.11), summing to 0.80.

$$\tilde{p}_1 = \frac{0.32}{0.80} = \mathbf{0.40}.$$

(For comparison, a fixed top- K sampler with, say, $K = 3$ would have missed the 4th token entirely, even though it still carries non-trivial probability mass 0.11.)

Problem N10 – Temperature scaling changes the distribution’s sharpness

Logits for 3 tokens are $u = (2.0, 1.0, 0.0)$. Compute the softmax probabilities at (a) $\mathcal{T} = 1$, (b) $\mathcal{T} = 0.5$ (low temperature), and (c) $\mathcal{T} = 2$ (high temperature). Comment on the trend.

Solution

(a) $\mathcal{T} = 1$: $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$; sum = 11.107.

$$p \approx (0.665, 0.245, 0.090).$$

(b) $\mathcal{T} = 0.5$: exponents become $u/\mathcal{T} = (4.0, 2.0, 0.0)$. $e^4 = 54.60$, $e^2 = 7.389$, $e^0 = 1$; sum = 63.0.

$$p \approx (0.867, 0.117, 0.016).$$

(c) $\mathcal{T} = 2$: exponents become $u/\mathcal{T} = (1.0, 0.5, 0.0)$. $e^1 = 2.718$, $e^{0.5} = 1.649$, $e^0 = 1$; sum = 5.367.

$$p \approx (0.507, 0.307, 0.186).$$

Trend: as $\mathcal{T}$ decreases from $2 \rightarrow 1 \rightarrow 0.5$, the top token's probability climbs $0.507 \rightarrow 0.665 \rightarrow 0.867$ – the distribution sharpens monotonically toward a one-hot arg max (greedy, in the $\mathcal{T} \rightarrow 0$ limit). As $\mathcal{T}$ increases, the three probabilities move toward each other (approaching the uniform $1/3$ each as $\mathcal{T} \rightarrow \infty$), confirming the low-temperature = less-random / high-temperature = more-random intuition numerically.

Problem N11 – Memory: attention scores and the KV cache

For GPT-1 ($n = 12$, $H = 12$, $d_{\text{model}} = 768$, $T = 512$) and one sequence ($B = 1$) stored in 16-bit floats (2 bytes per number): (a) How many bytes does the attention-score tensor $QK^\top$ take for *one* block? (b) How large is the KV cache at full context? (c) If the context were doubled to 1024, by what factor does each grow?

Solution

(a) Scores have shape $(H, T, T) = (12, 512, 512)$: $12 \times 512^2 = 3,145,728$ numbers $\times 2$ bytes = 6,291,456 bytes ≈ 6.3 MB per block.

(b) Keys and values for every block and position: $2 \times n \times T \times d_{\text{model}} = 2 \times 12 \times 512 \times 768 = 9,437,184$ numbers $\times 2$ bytes ≈ 18.9 MB.

(c) Scores scale with T^2 : $\times 4$ (to ≈ 25.2 MB per block). The KV cache scales with T : $\times 2$ (to ≈ 37.7 MB). This asymmetry is why long-context work focuses on the T^2 attention term during training and prefill, and on KV-cache size during generation.

6.2 Subjective Questions (with Full Answers)

Try to answer each question aloud or on paper before reading the green box under it.

Q1. Why is Byte Pair Encoding (BPE) used instead of whole-word tokenization for GPT's vocabulary?

Answer & Explanation

Whole-word tokenization needs a vocabulary large enough to cover (almost) every distinct word form, yet still fails on out-of-vocabulary words (rare words, typos, new proper nouns, morphological variants). BPE instead builds a vocabulary of frequently co-occurring *subword* units by iteratively merging the most frequent adjacent symbol pairs, starting from individual bytes/characters. This gives a fixed, moderate-sized vocabulary ($|\mathcal{V}| = 40,478$ for GPT-1) that can still represent *any* string (in the worst case, falling back to individual characters/bytes), while common words remain single tokens and rare/unseen words are decomposed into meaningful, previously-seen sub-word pieces. This directly controls both the embedding-matrix size ($|\mathcal{V}| \times d_{\text{model}}$) and the softmax output size.

Q2. Why does GPT use only masked self-attention and drop cross-attention entirely, even though it is derived from the Transformer’s “decoder”?

Answer & Explanation

Cross-attention exists specifically to let the decoder attend to an *encoder’s* output (e.g. the source-language representation in translation). GPT has no encoder at all – it is a single autoregressive stream trained purely on the next-token prediction objective over one sequence – so there is nothing for a cross-attention layer to attend to. Removing it also reduces parameters and simplifies the architecture to exactly what is needed: masked self-attention (to preserve causality while mixing information across positions already seen) followed by a position-wise FFN.

Q3. Why are positional embeddings *learned* in GPT-1 rather than the fixed sinusoidal embeddings of the original Transformer? What is the trade-off?

Answer & Explanation

Fixed sinusoidal embeddings are computed from a closed-form function of position and frequency, require no extra parameters, and (in principle) generalize to sequence lengths beyond those seen in training, since the function is defined for any position. Learned positional embeddings, by contrast, are simply a lookup table of shape (T, d_{model}) trained jointly with the rest of the network. The trade-off: learned embeddings can adapt to whatever positional patterns are actually useful for the training data/task (potentially better performance within the trained context length), at the cost of (a) a modest number of extra parameters ($512 \times 768 \approx 0.39\text{M}$ for GPT-1) and (b) an inability to gracefully extrapolate beyond the maximum trained context length T (position 513 simply has no learned embedding).

Q4. Why is the representation at the *last* token position used for the classification head during fine-tuning, rather than, say, averaging over all positions or using the first token?

Answer & Explanation

Because GPT uses *causal* (left-to-right) self-attention, the hidden state at position i has only attended to tokens $1, \dots, i$ – it has no information about anything after it. Only the final position (the $\langle e \rangle$ token) has attended to the *entire* input sequence, so it is the unique position whose representation summarizes the whole sentence. Averaging over all positions would mix in “partial-context” representations from earlier positions that never saw the full sentence, diluting the signal; using the first token ($\langle s \rangle$) would be even worse, since it has seen essentially nothing. (This is in contrast to bidirectional encoder models like BERT, where every position – including a prepended [CLS] token – can see the entire sequence, which is why BERT can use the *first* token for pooling.)

Q5. Explain teacher forcing and why it introduces “exposure bias.”

Answer & Explanation

Teacher forcing means that, during training, the ground-truth previous tokens (not the model’s own predictions) are fed in as the conditioning context at every position, for every step, simultaneously (enabled by the causal mask, since future ground-truth tokens are masked out of the computation anyway). This makes training fast (one parallel forward pass covers all T positions) and numerically stable (no compounding of the model’s own early mistakes during training). However, at *inference* time there is no ground truth – the

model must condition on tokens *it itself generated*. If the model produces a slightly-off token early in generation, all subsequent predictions are conditioned on that “wrong” history, a mismatch between training-time and inference-time input distributions known as **exposure bias**, which teacher forcing does not, by itself, correct for.

- Q6. Using the sentiment-neuron result, explain why unsupervised pre-training on raw text produces representations useful for downstream tasks with very little labelled data.**

Answer & Explanation

To predict the next token/character accurately across millions of diverse documents, a model is forced to implicitly capture a great deal of structure about language – syntax, topic, and, crucially, sentiment/opinion, since sentiment strongly influences which words are likely to follow (e.g., a negative review is more likely to be followed by words like “disappointing,” “waste,” etc.). The Radford et al. result showed this indirectly-learned structure could be so well organized that a *single* neuron’s value already separated positive from negative reviews almost perfectly, letting a simple linear classifier reach near state-of-the-art accuracy with orders of magnitude less labelled data than a fully supervised model trained from scratch. This is the core justification for the pre-train → fine-tune paradigm: the expensive, large-scale learning happens once, unsupervised, and the cheap, small-scale fine-tuning only needs to “read off” or lightly adjust already-useful representations.

- Q7. Compare encoder-only, decoder-only, and encoder-decoder pre-training objectives. When would each be preferred?**

Answer & Explanation

Encoder-only (bidirectional, masked-token prediction) produces representations that use full (left *and* right) context at every position – well suited to *understanding* tasks like classification, extractive QA, and token tagging, where the whole input is available at inference time and no left-to-right generation is required. *Decoder-only* (causal, next-token prediction) is naturally suited to *generation* tasks – open-ended text generation, dialogue, completion – since it is trained under exactly the same autoregressive process used at inference, and (as GPT shows) can be adapted to many understanding tasks too via input-formatting tricks. *Encoder-decoder* (bidirectional encoder + causal decoder with cross-attention) is best suited to *sequence-to-sequence* tasks with a clear input/output split of different (possibly very different) lengths and vocabularies – machine translation, summarization – where the decoder benefits from a full, bidirectionally-processed representation of the input to cross-attend to, while still generating its own output autoregressively.

- Q8. Why is a delimiter token like $\langle \$ \rangle$ necessary when fine-tuning GPT on sentence-pair tasks (e.g. entailment), instead of simply concatenating the two sentences?**

Answer & Explanation

Without an explicit marker, the model would see one undifferentiated run of tokens and would have to *implicitly* guess where one sentence ends and the next begins – unreliable, especially since natural sentences can end with all sorts of tokens. The delimiter is a dedicated, learned embedding whose entire “job” is to mark a segment boundary; the self-attention layers can learn to treat it as a structural signal (much like punctuation, but reserved and unambiguous), letting the model reliably distinguish “premise” tokens from “hypothesis” tokens within the single concatenated sequence that a decoder-only architecture requires.

Q9. Why is $-\infty$ used in the mask matrix (rather than, say, 0), and why is a large negative finite number used in practice instead of literal $-\infty$?

Answer & Explanation

The mask is added to the raw attention scores *before* the softmax. Softmax turns each masked score into a weight via $e^{\text{score}} / \sum e^{\text{score}}$. Using $-\infty$ guarantees $e^{-\infty} = 0$, so a masked position’s attention weight is *exactly* zero, with the remaining probability mass distributed only over the allowed positions in that row. Adding 0 (i.e., no mask) would leave the future positions fully visible, defeating the purpose. In practice, computers cannot represent literal $-\infty$ cleanly through floating-point arithmetic operations – an entire row of $-\infty$ before any exponentiation, or intermediate NaN/Inf arithmetic, can produce numerically unstable NaNs. Frameworks instead use a large-magnitude negative finite number (e.g., -10^9 or the dtype’s minimum representable value), which, after exponentiation, underflows to a floating-point 0 just as effectively, without the numerical hazards of true infinities.

Q10. Why might GELU outperform ReLU in transformer architectures, and what is the intuition behind its “stochastic gate” interpretation?

Answer & Explanation

$\text{ReLU}(x) = \max(0, x)$ is piecewise linear with a sharp, non-differentiable corner at $x = 0$: it deterministically either passes x through unchanged or kills it completely. $\text{GELU}(x) = x \cdot \Phi(x)$ (with Φ the standard normal CDF) can be interpreted as multiplying x by the *probability* that a standard normal random variable is less than x – i.e., a smooth, probabilistic (“soft”) gate rather than a hard 0/1 switch. This makes GELU smooth and differentiable everywhere (better-behaved gradients, particularly around zero) and allows small negative inputs to pass through with a small negative output (like ELU), rather than being hard-clipped to zero – empirically this has been found to improve optimization and final performance in deep transformer stacks, which is why GPT (and most subsequent transformer LMs) adopted it over ReLU.

Q11. Why is greedy search not guaranteed to find the overall most probable sequence, even though it picks the “best” token at every single step?

Answer & Explanation

Sequence probability is a *product* over the whole path, $P(x_1, \dots, x_T) = \prod_i P(x_i | x_{<i})$, and every one of GPT-1’s conditional distributions is itself a function of the *entire* prefix chosen so far. Greedy search commits to $\arg \max_x P(x | x_{<i})$ at step i *without any knowledge of how that choice will affect* $P(x_{i+1} | x_{\leq i}), P(x_{i+2} | \cdot), \dots$. A token that looks locally optimal at step i can lead into a region of the sequence space where all subsequent conditionals are mediocre, while a locally *second*-best token at step i might open up a path where every subsequent conditional is very high – and because probabilities multiply, a strong subsequent path can easily overcome one weaker early factor. This is a textbook instance of a **greedy algorithm failing to solve a problem that requires global optimization** (the general lesson from algorithms: local optimality does not imply global optimality unless the problem has special structure, e.g. the matroid/exchange property, which sequence generation does not have).

Q12. Explain, intuitively, why beam search with a larger beam width k produces *more* deterministic-feeling, safer text, not less.

Answer & Explanation

This can feel counterintuitive (“more options tracked $\Rightarrow$ more variety?”), but beam search still always *outputs a single sequence* – the highest-scoring one among the k survivors – so it is exactly as deterministic as greedy search ($k = 1$) for a fixed prompt and model. What a *larger* k changes is only how close that single output gets to the true global optimum (exhaustive search’s answer): a bigger beam is a *better search*, not a source of randomness. Since the true global optimum is itself typically a “safe,” high-average-probability sequence (that is, after all, what “maximum probability” means), searching *harder* for it (larger k) tends to converge on *more* of exactly the same generic, unsurprising, low-variance kind of text discussed in the beam-vs-human surprise plot – not less. Diversity requires abandoning the “find the highest-probability sequence” objective altogether in favor of *sampling*, which is why stochastic decoding (top- K /top- p), not a bigger beam, is the fix for degeneracy/lack of creativity.

Q13. Connect the “surprise = uncertainty” observation to Shannon entropy. Why does this explain why purely deterministic decoding produces bland text?

Answer & Explanation

The *self-information* (“surprise”) of observing a particular outcome x with probability $p(x)$ is defined as $-\log p(x)$: an outcome that was near-certain ($p(x) \approx 1$) carries almost no surprise ($-\log p(x) \approx 0$), while a rare outcome ($p(x) \rightarrow 0$) carries large surprise ($-\log p(x) \rightarrow \infty$). The *expected* surprise over a whole distribution is exactly the **Shannon entropy**, $H = -\sum_x p(x) \log p(x)$. Greedy and beam search are explicitly designed to *maximize* the probability of the chosen path at every step – i.e., they systematically seek out the *lowest-surprise*, lowest-entropy choices available. Human-written text, by contrast, empirically has much higher per-step entropy/variance (the “Human” curve in the surprise plot swings between low and high probability constantly), because natural language routinely uses less-predictable-but-still-valid word choices for emphasis, specificity, or style. A decoding strategy that only ever picks near-certain tokens is, by construction, choosing the *lowest-entropy* path through the sequence space at every step – which is precisely why it reads as generic/bland rather than natural.

Q14. Why does top- p (nucleus) sampling solve the fixed- K problem that plain top- K sampling has, without needing to manually tune K per example?

Answer & Explanation

Top- K sampling fixes the *count* of eligible tokens regardless of how the probability mass is actually distributed among them at that particular step. Top- p instead fixes the *amount of probability mass* that must be covered, and lets the *count* of tokens needed to reach it float freely. On a flat/uncertain step (many plausible continuations, each with modest probability), reaching a cumulative mass of p mechanically requires *more* tokens, so the nucleus automatically grows and preserves diversity. On a peaked/confident step (one or two tokens dominate), the same cumulative-mass threshold p is reached after just one or two tokens, so the nucleus automatically shrinks and avoids pulling in near-noise options. In other words, top- p directly targets the quantity we actually care about (*how much of the model’s belief are we sampling from*) rather than a proxy for it (*how many tokens*), so it adapts to the model’s per-step confidence without any external tuning.

6.3 Multiple Choice Questions

Attempt all 25 questions before looking at the answer key (Section 6.4). Questions 1–15 cover Modules 1–3; questions 16–25 cover Module 4.

MCQ 1. By the chain rule of probability, $P(x_1, x_2, x_3)$ equals:

- (a) $P(x_1)P(x_2)P(x_3)$
- (b) $P(x_1) P(x_2 \mid x_1) P(x_3 \mid x_1, x_2)$
- (c) $P(x_3) P(x_2 \mid x_3) P(x_1 \mid x_2, x_3)$
- (d) $P(x_1, x_2) + P(x_3)$

MCQ 2. Which component is present in a GPT “transformer block” but *absent* compared to a full sequence-to-sequence transformer’s decoder layer?

- (a) Masked multi-head self-attention
- (b) Feed-forward network
- (c) Multi-head cross-attention
- (d) Residual connections

MCQ 3. In the causal mask M , what value is placed at position (i, j) when $j > i$ (a future position relative to the query)?

- (a) 0
- (b) 1
- (c) $-\infty$ (in practice, a large negative number)
- (d) $+\infty$

MCQ 4. GPT-1’s per-head query/key/value projection matrices have shape:

- (a) 768×768
- (b) 768×64
- (c) 64×64
- (d) 512×768

MCQ 5. Which of the following is *closest* to GPT-1’s FFN hidden layer size?

- (a) 768
- (b) 1536
- (c) 3072
- (d) 40,478

MCQ 6. GPT-1 was pre-trained on which corpus?

- (a) Common Crawl
- (b) BookCorpus
- (c) Wikipedia only
- (d) The 1B Word Benchmark (used exclusively)

MCQ 7. What tokenizer does GPT-1 use to build its 40,478-token vocabulary?

- (a) WordPiece
- (b) Character-level tokenization
- (c) Byte Pair Encoding (BPE)
- (d) SentencePiece unigram model

MCQ 8. GPT-1 uses which activation function inside its feed-forward network?

- (a) ReLU
- (b) Sigmoid
- (c) Tanh
- (d) GELU

MCQ 9. Why does pre-training use **teacher forcing** rather than free-running autoregressive generation?

- (a) It uses less GPU memory
- (b) It enables faster, parallel, more stable training over all positions at once
- (c) It removes the need for a causal mask
- (d) It guarantees zero exposure bias at inference

MCQ 10. During fine-tuning for classification, which hidden state is fed into the new task head W_y ?

- (a) The average of all position representations
- (b) The representation at the first token, $\langle s \rangle$
- (c) The representation at the last position of the last layer, h_l^m
- (d) The token embedding matrix directly

MCQ 11. In fine-tuning for textual entailment, what is the role of the token $\langle \$ \rangle$?

- (a) It marks the end of the entire sequence
- (b) It is a padding token
- (c) It delimits/separates the two input segments (e.g. premise and hypothesis)
- (d) It replaces the softmax output layer

MCQ 12. In the multiple-choice fine-tuning setup with k choices, how are the final answer probabilities obtained?

- (a) Each choice's score is independently passed through its own sigmoid
- (b) All k choices are concatenated into a single input sequence and classified once
- (c) Each choice gets its own forward pass through the shared transformer, a linear layer turns each last-position vector into a score, and the k scores are jointly normalized with one softmax
- (d) Only the highest-scoring choice's transformer pass is used; the rest are discarded before scoring

MCQ 13. Which of the following is *not* listed as part of the “wishlist” for text generation in these notes?

- (a) Discourage degenerate (repetitive/incoherent) text
- (b) Encourage creativity/diversity across generations
- (c) Accelerate the generation of tokens
- (d) Guarantee the output is factually correct

MCQ 14. What stops the autoregressive text-generation loop in the worked example?

- (a) Only reaching a fixed maximum sequence length
- (b) Only generating the $\langle e \rangle$ token
- (c) Either reaching a maximum sequence length *or* generating $\langle e \rangle$, whichever comes first

- (d) The loss falling below a threshold

MCQ 15. Positional embeddings in GPT-1, compared to the original Transformer, are:

- (a) Fixed sinusoidal functions of position, exactly as in the original Transformer
- (b) Learned parameters, unlike the original Transformer’s fixed sinusoidal encoding
- (c) Not used at all
- (d) Computed from the token embeddings via a linear layer

MCQ 16. Exhaustive decoding search over a sequence of length T with vocabulary size $|\mathcal{V}|$ must compare how many candidate sequences?

- (a) $O(T)$
- (b) $O(|\mathcal{V}| \cdot T)$
- (c) $O(|\mathcal{V}|^T)$
- (d) $O(\log |\mathcal{V}|)$

MCQ 17. What is the key limitation the “pick the second-best token at step 1” example is meant to demonstrate about greedy search?

- (a) Greedy search is slower than beam search
- (b) Greedy search’s step-by-step locally-best choices do not always yield the globally highest-probability sequence
- (c) Greedy search cannot be implemented with a causal mask
- (d) Greedy search requires more parameters than beam search

MCQ 18. In beam search with beam width k , how many candidate continuations must be scored at each time step (before pruning back down to k)?

- (a) k
- (b) $|\mathcal{V}|$
- (c) $k \times |\mathcal{V}|$
- (d) $k^{|\mathcal{V}|}$

MCQ 19. Setting beam width $k = 1$ makes beam search equivalent to:

- (a) Exhaustive search
- (b) Greedy search
- (c) Top- K sampling with $K = 1$
- (d) Top- p sampling with $p = 1$

MCQ 20. Why is length normalization (dividing log-probability by sequence length) necessary in beam search?

- (a) To make the computation run faster
- (b) Because otherwise longer sequences are mechanically penalized, since they are products of more sub-1 probability factors
- (c) To convert probabilities into logits
- (d) It is only needed for greedy search, not beam search

MCQ 21. According to the “beam search vs. human” probability-over-time plot, which statement is correct?

- (a) Beam search text has high variance in per-token probability; human text has low variance
- (b) Both beam search and human text have similarly high variance
- (c) Beam search text has low variance (stays near probability 1); human text has high variance, frequently dipping to low probability
- (d) Neither beam search nor human text shows any consistent pattern

MCQ 22. In top- K sampling, what must happen to the top- K raw probabilities *before* a token is sampled?

- (a) Nothing – sample directly using the raw probabilities
- (b) They must be renormalized (divided by their own sum) so they sum to 1
- (c) They must be converted back into logits
- (d) They must all be set equal to $1/K$

MCQ 23. Why can a single fixed value of K be a poor choice for top- K sampling across different generation steps?

- (a) K must always equal the vocabulary size
- (b) A small fixed K excludes valid options on “flat” (uncertain) steps, while the same K can include near-noise tokens on “peaked” (confident) steps
- (c) Fixed K makes decoding non-differentiable
- (d) K has no effect on the output distribution

MCQ 24. As temperature $\mathcal{T} \rightarrow 0^+$ in temperature-scaled softmax sampling, the sampling distribution approaches:

- (a) A uniform distribution over the vocabulary
- (b) A one-hot distribution on the arg max token (i.e., greedy decoding)
- (c) The original untouched softmax ($\mathcal{T} = 1$)
- (d) An undefined / NaN distribution in all cases

MCQ 25. In top- p (nucleus) sampling with $p = 0.6$ and sorted probabilities (0.5, 0.3, 0.15, 0.05), which tokens form the nucleus?

- (a) Only the first token (0.5)
- (b) The first two tokens (0.5, 0.3)
- (c) The first three tokens (0.5, 0.3, 0.15)
- (d) All four tokens

6.4 MCQ Answer Key with Explanations

- MCQ 1. (b).** The chain rule factors the joint distribution left-to-right into conditionals, each conditioned on all *previous* variables: $P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$. (a) is the incorrect independence assumption; (c) reverses the order; (d) is not a valid probability identity.
- MCQ 2. (c).** GPT has no encoder, so there is nothing to cross-attend to; the cross-attention sub-layer is dropped entirely. Masked self-attention, the FFN, and residual connections are all retained.
- MCQ 3. (c).** This makes the post-softmax attention weight for future positions exactly (or effectively) 0, since $e^{-\infty} = 0$.
- MCQ 4. (b).** $d_{\text{model}} = 768$, $H = 12$ heads $\Rightarrow d_k = d_{\text{model}}/H = 64$; each of W_Q, W_K, W_V maps $d_{\text{model}} \rightarrow d_k$, i.e. shape 768×64 .
- MCQ 5. (c).** $d_{\text{ff}} = 4 \times d_{\text{model}} = 4 \times 768 = 3072$.
- MCQ 6. (b).** BookCorpus (7,000 books, 74M sentences, ≈ 1 B words, 16 genres, long-range contiguous text). The 1B Word Benchmark is mentioned only as an alternative whose sentences are *not* contiguous.
- MCQ 7. (c).** Byte Pair Encoding (BPE), as explicitly stated in the input-data slide.
- MCQ 8. (d).** GELU (Gaussian Error Linear Unit) – smooth and non-monotonic near zero, unlike ReLU.
- MCQ 9. (b).** Feeding ground-truth previous tokens lets every position of a sequence be trained in one parallel forward pass (with the causal mask still required to prevent look-ahead); it does *not* eliminate exposure bias – that mismatch appears specifically *because* training uses teacher forcing while inference cannot.
- MCQ 10. (c).** Because of causal masking, only the last position has attended to the full input sequence, making h_l^m (last layer, last time step) the appropriate summary representation.
- MCQ 11. (c).** $\langle \$ \rangle$ is a delimiter placed between the two concatenated segments so the model can structurally distinguish premise from hypothesis within one input stream.
- MCQ 12. (c).** The transformer weights are shared across the k passes (one per choice); a linear layer (drawn as *Linear-i* on the slide, the same weights in GPT-1) turns each pass’s last-position vector into a scalar score, and *all* k scores together are normalized with a single softmax to produce the final distribution over choices.
- MCQ 13. (d).** Factual correctness is not part of the stated wishlist – the three listed goals are avoiding degeneracy, encouraging creativity/diversity, and speeding up generation.
- MCQ 14. (c).** The example uses *both* criteria simultaneously: stop at a maximum sequence length (e.g. 5) *or* as soon as the end token $\langle e \rangle$ is generated.
- MCQ 15. (b).** GPT-1 learns a positional embedding matrix of shape (context length $\times d_{\text{model}}$) = 512×768 , jointly with the rest of the network, rather than using the fixed sinusoidal scheme.
- MCQ 16. (c).** Each of the T positions multiplies the number of candidates under consideration by $|\mathcal{V}|$, giving an exponential $O(|\mathcal{V}|^T)$ total – intractable for realistic vocabulary sizes, which is exactly why greedy/beam search exist.
- MCQ 17. (b).** The worked example shows a sequence built by taking the *second*-best first token achieving a higher overall sequence probability (0.035) than the fully-greedy sequence (0.011) – proof that greedy’s step-wise local optimality does not guarantee global optimality.

- MCQ 18. (c).** Each of the k surviving beams is extended by every one of the $|\mathcal{V}|$ vocabulary tokens, giving $k \times |\mathcal{V}|$ candidates, which are then pruned back down to the top- k by cumulative score.
- MCQ 19. (b).** With only one beam kept at every step, beam search always extends the single current best partial sequence by its single best next token – exactly greedy search. (Note (c) is a near-miss: top- K with $K = 1$ is also degenerate/greedy in effect, but the question specifically asks about beam search’s own parameter.)
- MCQ 20. (b).** Every conditional factor $P(x_i \mid x_{<i}) \leq 1$, so a sequence’s total probability strictly shrinks as more factors are multiplied in – purely as an artifact of length, not quality. Normalizing by length allows fair comparison across candidate sequences of different lengths.
- MCQ 21. (c).** Beam search’s chosen tokens stay close to probability 1 at almost every step (low variance – it always plays it safe), while real human-written text frequently chooses much less predictable, lower-probability words (high variance) – this gap is exactly what makes deterministic search output feel comparatively generic.
- MCQ 22. (b).** The top- K raw probabilities are a subset of a distribution that summed to 1 over the *whole* vocabulary; restricting to K tokens leaves them summing to less than 1, so they must be renormalized by their own sum to form a valid probability distribution to sample from.
- MCQ 23. (b).** The shape of the next-token distribution varies step to step: on a flat distribution a fixed small K misses plausible continuations, while on a peaked distribution the same K can admit low-probability, nearly meaningless tokens – this is precisely the motivation for temperature scaling and, especially, top- p sampling.
- MCQ 24. (b).** Dividing logits by a vanishingly small $\mathcal{T}$ makes the largest logit’s exponential dominate every other term arbitrarily strongly, so the softmax output collapses onto a one-hot vector at the arg max – i.e., temperature-0 sampling behaves exactly like greedy search.
- MCQ 25. (b).** Cumulative sum: 0.5 (not yet > 0.6), then $0.5 + 0.3 = 0.8$ which *exceeds* 0.6 – so the nucleus stops growing right after the second token, giving $\{0.5, 0.3\}$ (renormalized to sum to 1 before sampling).